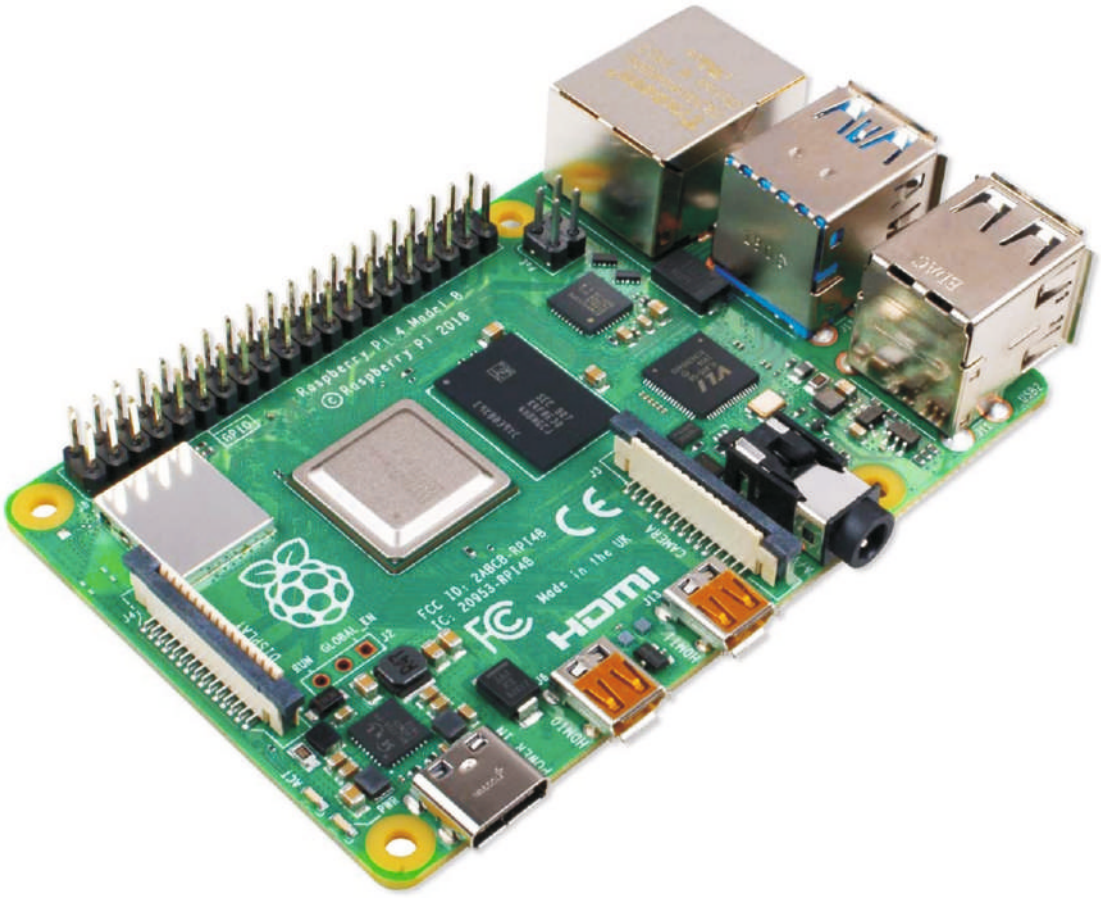
EDKITS ELECTRONICS

Raspberry Pi Guide for TYBSc IT



Contact: 9869059884.

Email: edkits@gmail.com

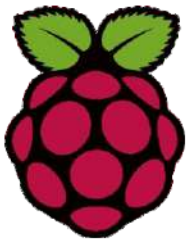


Table of Contents

0) Raspberry Pi Hardware Preparation and OS Installation & Familiarizing with Pi 3

0a) GPIO: Light the LED with Python 6

1) Displaying different LED Patterns with Raspberry Pi 10

2) Displaying Time over 4 Digit 7 Segment Display using Raspberry Pi 13

3) Interfacing 16x2 LCD (I2C) Display with Raspberry Pi 16

4) Raspberry Pi based Oscilloscope 19

5) Controlling Raspberry Pi with Telegram 25

6) Fingerprint Sensor interfacing with Raspberry Pi using onboard UART 27

6a) Fingerprint Sensor interfacing with Raspberry Pi using USB to TTL Converter 35

7) Raspberry Pi GPS Module Interfacing ....., ... 41

8) IOT Based Home Automation using raspberry Pi 47

9) Interfacing Pi Camera with Raspberry Pi 51

10) Interfacing Raspberry Pi with RFID RC522 Chip 54

11) Installing Windows 10 IOT Core on Raspberry Pi 62

12) Interface Raspberry Pi with DC Motor with Driver L293D..... 64

13) Raspberry Pi based Oscilloscope (for Bulls eye OS) 66

EDKITS

Raspberry Pi Hardware Preparation and Installation

Hardware Guide:

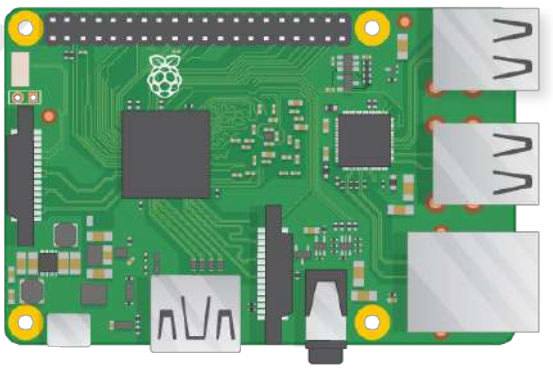
For getting started with raspberry pi for the first time you will require the following hardware

1. Raspberry Pi (latest Model)
2. Monitor or TV
3. HDMI cable
4. Ethernet cable
5. USB keyboard
6. USB mouse
7. Micro USB power supply
8. 8GB or larger microSD card
9. SD Card Reader

Raspberry Pi 3 Model B:

The Raspberry Pi 3 is the third generation Raspberry Pi. It replaced the Raspberry Pi 2 Model B in February 2016. Compared to the Raspberry Pi 2 it has:

- A 1.2GHz 64-bit quad-core ARMv8 CPU
- 802.11n Wireless LAN
- Bluetooth 4.1
- Bluetooth Low Energy (BLE)



Like the Pi 2, it also has:

- 4 USB ports
- 40 GPIO pins
- Full HDMI port
- Ethernet port
- Combined 3.5mm audio jack and composite video
- Camera interface (CSI)
- Display interface (DSI)
- Micro SD card slot (now push-pull rather than push-push)
- Video Core IV 3D graphics core
- The Raspberry Pi 3 has an identical form factor to the previous Pi 2 (and Pi 1 Model B+) and has complete compatibility with Raspberry Pi 1 and 2.

EDKITS

Monitor or TV:

A monitor or TV with HDMI in can be used as a display with a Raspberry Pi. Most modern television sets and monitors have an HDMI port, and are the easiest to get working with t he Raspberry Pi. You can use an HDMI cable to connect the Raspberry Pi directly to the television or monitor.

Some older monitors have a DVI port. These work well with the Raspberry Pi, although you'll need an HDMI-to-DVI adapter to attach to an HDMI cable, or a one-piece HDMI-to-DVI cable. Some old monitors have a VGA port. These can be trickier to use as you'll need an HDMI-to-VGA converter, which can change digital video to analogue video. A simple port adapter won't work.

HDMI to HDMI Cable:

Connect Raspberry Pi to a Monitor or TV with a HDMI to HDMI cable.

Ethernet cable:

Ethernet cable will allow your Pi to connect with the internet. It is also useful for headless setup of Raspberry Pi

USB Keyboard and Mouse:

Any standard USB keyboard and mouse can be used with the Raspberry Pi. This plug and play devices will work without any additional driver. Simply plug them into the Raspberry Pi and they should be recognised when it starts up.

Power Supply:

It is recommended that you use a 5V, 2A USB power supply for all models of Raspberry Pi.

SD Card:

The latest version of Raspbian, the default operating system recommended for the Raspberry Pi, requires an 8GB (or larger) micro SD card. SD card will store the operating systems as well as all the file and applications created by you.

Installation Guide:

Now since you have all the required hardware, we will now learn how to get the operating system onto your microSD card so that you can start using software on your Raspberry Pi

Get Raspbian OS on your microSD card:

Raspbian comes pre-installed with plenty of software for education, programming and general use. It has Python, Scratch, Sonic Pi, Java, Mathematica and more.

EDKITS

1. To download Raspbian log on to raspberrpi.org and click on the download, then click on Raspbian and lastly download the RASPBIAN JESSIE WITH DESKTOP file. You can choose either the Torrent file or ZIP file.
2. The downloaded file will be in zip format. To unzip the file, you will require an unzip tool. You can use any unzipping tool viz. WINRA R, 7ZIP etc. After unzipping the file, you will find a disc image file in the unzipped folder.
3. Now format the SD Card before writing the disc image file on the SD card. You can use SD Formatter tool or any other tool of your wish.
4. To write the image file o f the operating system on the SD card you will require a Disk Imager tool. For this you can use Win32 Disk Imager tool.
5. Once the image is written on the SD Card, your untitled SD card will now have the name boot. Your SD Card will now hold the Raspbian Op erating system required for the first -time setup.

Plugging in your Raspberry Pi:

1. Begin by placing your SD card into the SD card slot on the Raspberry Pi. It will only fit one way.
2. Next, plug your keyboard and mouse into the USB ports on the Raspberry Pi.
3. Make sure that your monitor or TV is turned on, and that you have selected the right input (e.g. HDMI 1, DVI, etc).
4. Connect your HDMI cable from your Raspberry Pi to your monitor or TV.
5. If you intend to connect your Raspberry Pi to the internet, plug an E thernet cable into the Ethernet port, or connect a WiFi dongle to one of the USB ports (unless you have a Raspberry Pi 3).
6. When you're happy that you have plugged all the cables and SD card in correctly, connect the micro USB power supply. This action will turn on and boot your Raspberry Pi.

NOTE: for Raspberry Pi 4 please follow the instruction given below.

1. To download Raspberry pi OS (buster)log on to **raspberrypi.com** and click on the download and then click of rasberry pi os and click on **Raspberry pi OS buster.**

2. Follow the above mention step from 2 to 5 of **INSTALLATION GUIDE** section.

3. After writing the image on SD Card plug in the SD Card in to the Raspberry pi 4. If boot normally and you see the raspberry pi screen than ok. But if not than make the following changes in the SD card ‘config' file with notepad++.

1. Uncomment the line ‘hdmii_safe=1’
2. Uncomment the line 'hdmi_force_hotplug=1’
3. Uncomment the line 'hdmi_group=1’
4. Uncomment the line 'hdmi_mode=1'
5. Save and exit the 'config' file
6. Now insert again the memory card into the raspberry pi it will work.

EDKITS

GPIO: Light the LED with Python

After setting up the raspberry pi and having hands on practice with the Linux commands, you are now familiar with raspberry pi. Now it's time to work with the GPIO pins of the raspberry pi to have an external interface with the raspberry pi.

Hardware Guide:

Along with the basic setup you will require the following components to get started with the GPIO pins as follows:

1. LED
2. Resistor
3. Connecting wires
4. Breadboard

Before learning this lesson, you must understand the pin numbering system of the GPIO pins.

GPIO?

One powerful feature of the Raspberry Pi is the row of GPIO (general purpose input/output) pins along the top edge of the board.

These pins are a physical interface between the Pi and the outside world. At the simplest level, you can think of them as switches that you can turn on or off (input) or that the Pi can turn on or off (output). Of the 40 pins, 26 are GPIO pins and the others are power or ground pins (plus two ID EEPROM pins which you should not play with unless you know your stuff!)



What are they for? What can we do with them?

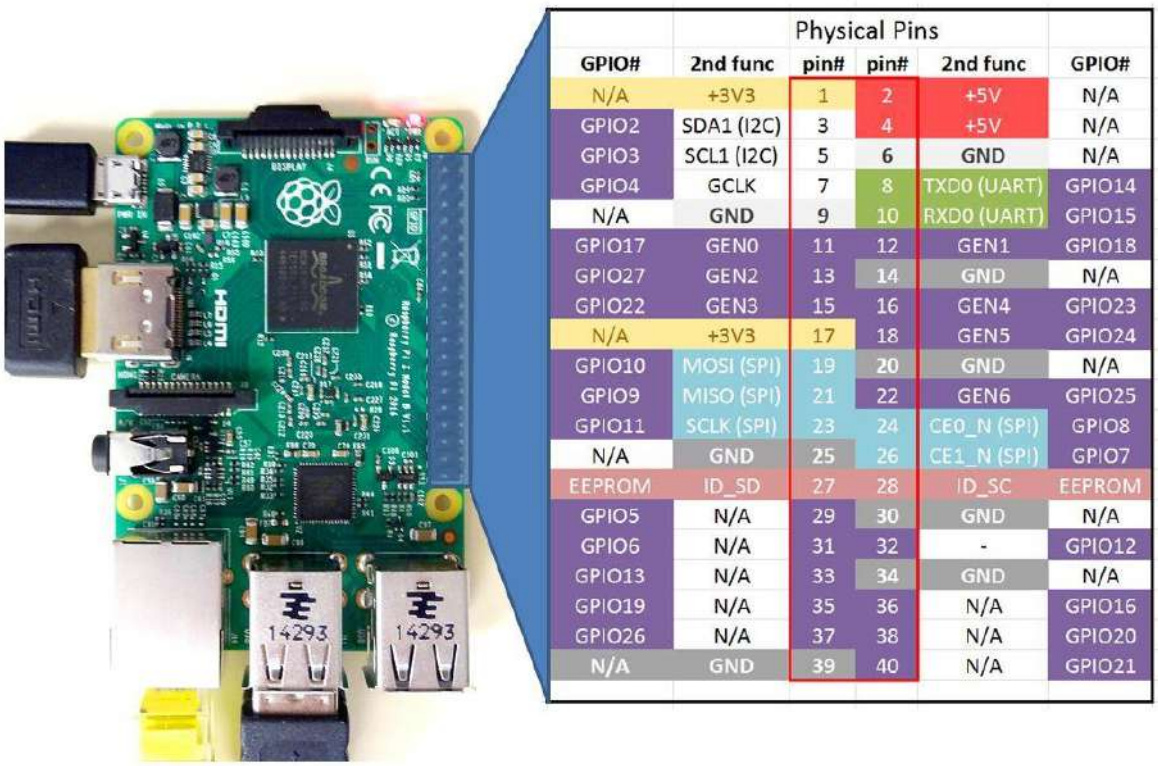
You can program the pins to interact in amazing ways with the real world. Inputs don't have to come from a physical switch; it could be input from a sensor or a signal from another computer or device, for example. The output can also do anything, from turning on an LED to sending a signal or data to another device. If the Raspberry Pi is on a network, you can control devices that are attached to it from anywhere** and those devices can send data back. Connectivity and control of physical devices over the internet is a powerful and exciting thing, and the Raspberry Pi is ideal for this.

How the GPIO pins work?

Output
When we use a GPIO pin as an output. Each pin can turn on or off, or go HIGH or LOW in computing terms. When the pin is HIGH it outputs 3.3 volts (3v3); when the pin is LOW it is off.

Input
GPIO outputs are easy; they are on or off, HIGH or LOW, 3v3 or 0v. Inputs a re a bit trickier because of the way that digital devices work. Although it might seem reasonable just to connect a button across an input pin and a ground pin, the Pi can get confused as to whether the button is on or off. It might work properly, it might not. It's a bit like floating about in deep space; without a reference, it would be hard to tell if you were going up or down, or even what up or down meant!
Therefore, you will see phrases like "pull up" and "pull down" in Raspberry Pi GPIO tutorials. It 's a way of giving the input pin a reference so it knows for certain when an input is received.

Warning: Randomly plugging wires and power sources into your Pi, however, may kill it. Bad things can also happen if you try to connect things to your Pi that use a lot of power.



		Physical Pins			
GPIO#	2nd func	pin#	pin#	2nd func	GPIO#
N/A	+3V3	1	2	+5V	N/A
GPIO2	SDA1 (I2C)	3	4	+5V	N/A
GPIO3	SCL1 (I2C)	5	6	GND	N/A
GPIO4	GCLK	7	8	TXD0 (UART)	GPIO14
N/A	GND	9	10	RXD0 (UART)	GPIO15
GPIO17	GEN0	11	12	GEN1	GPIO18
GPIO27	GEN2	13	14	GND	N/A
GPIO22	GEN3	15	16	GEN4	GPIO23
N/A	+3V3	17	18	GEN5	GPIO24
GPIO10	MOSI (SPI)	19	20	GND	N/A
GPIO9	MISO (SPI)	21	22	GEN6	GPIO25
GPIO11	SCLK (SPI)	23	24	CE0_N (SPI)	GPIO8
N/A	GND	25	26	CE1_N (SPI)	GPIO7
EEPROM	ID_SD	27	28	ID_SC	EEPROM
GPIO5	N/A	29	30	GND	N/A
GPIO6	N/A	31	32	-	GPIO12
GPIO13	N/A	33	34	GND	N/A
GPIO19	N/A	35	36	N/A	GPIO16
GPIO26	N/A	37	38	N/A	GPIO20
N/A	GND	39	40	N/A	GPIO21

A note on pin numbering:

When programming the GPIO pins there are two different ways to refer to them: GPIO numbering and physical numbering.

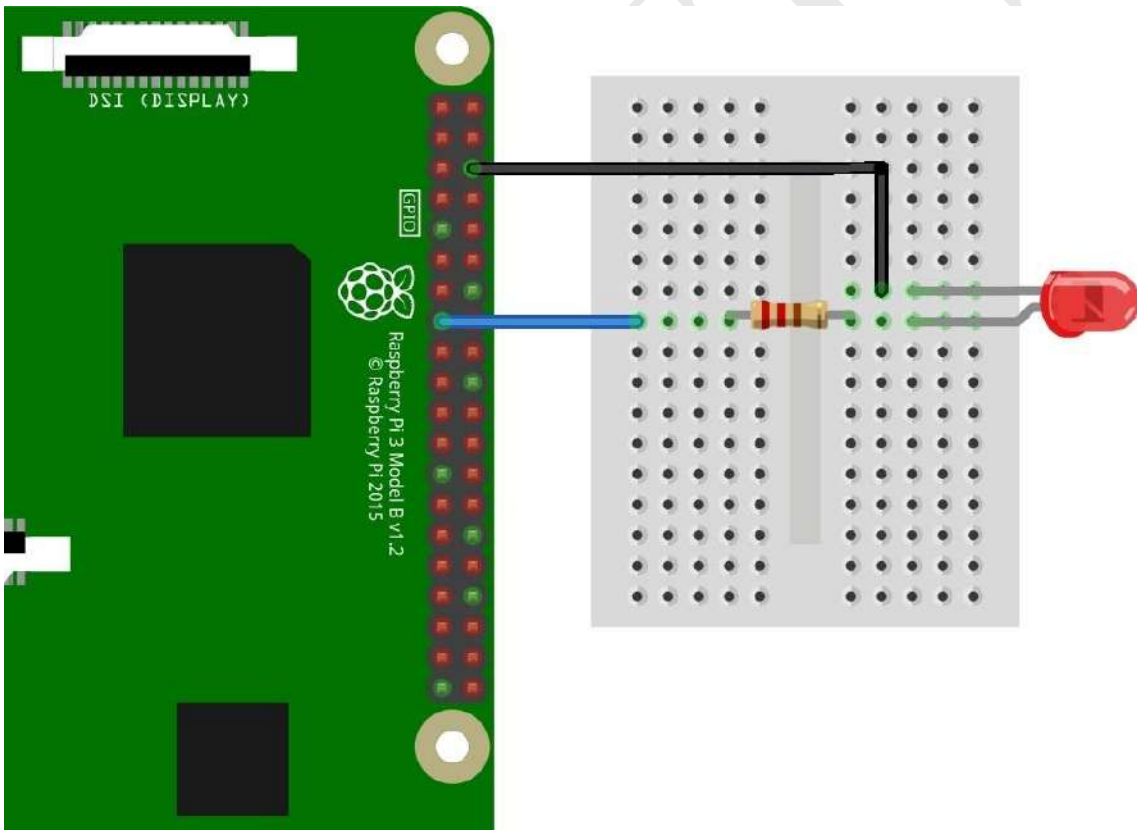
GPIO numbering:
These are the GPIO pins as the computer sees them. The numbers don't make any sense to humans, they jump about all over the place, so there is no easy way to remember them. You will need a printed reference or a reference board that fits over the pins.

Physical numbering:
The other way to refer to the pins is by simply counting across and down from pin 1 at the top left (nearest to the SD card).

Wiring up your Circuit:

1. Connect the GPIO22 (i.e. Physical Pin 15) Pin od raspberry pi to one end of the resistor.
2. Connect another end of resistor to the positive end (anode) of LED
3. Connect the negative end (cathode) of LED to Ground of raspberry pi.
4. Then Power on your raspberry pi

Circuit Diagram:



Software Guide:

Raspbian OS comes with many preinstalled programming environments. Here we will be using Python for coding.

To open Python, click on the application Menu, navigate to Programming, then click on Python 3 (IDLE) an Integrated Development Environment for Python 3. After opening the IDE, go to files and open new file to start your code.

Code:

```
#Blink LED Program
#Connect the LED to GPIO 22 Pin
#LED Blink Progarm
#Connect the LED to GPIO22 (i.e. Physical Pin15)


#import GPIO and time library
import RPi.GPIO as GPIO
from time import sleep

GPIO.setmode(GPIO.BCM)        #set the Pin mode you will be working with

ledPin = 22                   #this is GPIO22 pin i.e Physical Pin15

#setup the ledPin(i.e. GPIO22) as output
GPIO.setup(ledPin, GPIO.OUT)
GPIO.output(ledPin, False)

try:
  while True:
    GPIO.output(ledPin, True)  #Set the LED Pin to HIGH
    print("LED ON")
    sleep(1)                   #Wait for 1 sec
    GPIO.output(ledPin, False) #Set the LED Pin to LOW
    print("LED OFF")
    sleep(1)                   #wait for 1 sec
finally:
  #reset the GPIO Pins
  GPIO.output(ledPin, False)
  GPIO.cleanup()

#end of code
```

After writing your code save is on a desired location and run it to enjoy the fun.
Great! You have completed your first GPIO program. Now you are confident enough to go forward to explore and interface new things with raspberry Pi.
If you were unable to blink the LED, don't worry, recheck your connection and debug the error so that you don't repeat it again.
Now in the next lessons we will explore and interface new things

EDKITS

Displaying different LED Patterns with Raspberry P

For displaying Differen LED pattern connecte 8 LEDs in the same format to the pin number given in the below python Code :

CODE FOR LED PATTERN:

```
import RPi.GPIO as GPIO
import time
GPIO.setmode(GPIO.BOARD)
led1 = 29
led2 = 31
led3 = 33
led4 = 35
led5 = 36
led6 = 37
led7 = 38
led8 = 40
#setup the ledPin(i.e. GPIO22) as output
GPIO.setup(led1, GPIO.OUT)
GPIO.setup(led2, GPIO.OUT)
GPIO.setup(led3, GPIO.OUT)
GPIO.setup(led4, GPIO.OUT)
GPIO.setup(led5, GPIO.OUT)
GPIO.setup(led6, GPIO.OUT)
GPIO.setup(led7, GPIO.OUT)
GPIO.setup(led8, GPIO.OUT)
GPIO.output(led1, False)
GPIO.output(led2, False)
GPIO.output(led3, False)
GPIO.output(led4, False)
GPIO.output(led5, False)
GPIO.output(led6, False)
GPIO.output(led7, False)
GPIO.output(led8, False)

def ledpattern(ledVal1, ledVal2, ledVal3, ledVal4, ledVal5, ledVal6, ledVal7,ledVal8):
  GPIO.output(led1, ledVal1)
  GPIO.output(led2, ledVal2)
  GPIO.output(led3, ledVal3)
  GPIO.output(led4, ledVal4)
  GPIO.output(led5, ledVal5)
  GPIO.output(led6, ledVal6)
  GPIO.output(led7, ledVal7)
  GPIO.output(led8, ledVal8)
```

EDKITS

```
def patterOne():
  for i in range (0, 3):
    ledpattern(1, 0, 1, 0, 1, 0, 1, 0)
    time.sleep(1)
    ledpattern(0, 1, 0, 1, 0, 1, 0, 1)
    time.sleep(1)

def patternTwo():
  for i in range (0, 5):
    ledpattern(1, 0, 0, 0, 0, 0, 0, 0)
    time.sleep(0.1)
    ledpattern(0, 1, 0, 0, 0, 0, 0, 0)
    time.sleep(0.1)
    ledpattern(0, 0, 1, 0, 0, 0, 0, 0)
    time.sleep(0.1)
    ledpattern(0, 0, 0, 1, 0, 0, 0, 0)
    time.sleep(0.1)
    ledpattern(0, 0, 0, 0, 1, 0, 0, 0)
    time.sleep(0.1)
    ledpattern(0, 0, 0, 0, 0, 1, 0, 0)
    time.sleep(0.1)
    ledpattern(0, 0, 0, 0, 0, 0, 1, 0)
    time.sleep(0.1)
    ledpattern(0, 0, 0, 0, 0, 0, 0, 1)
    time.sleep(0.1)

def patternThree():
  for i in range (0, 5):
    ledpattern(0, 0, 0, 0, 0, 0, 0, 1)
    time.sleep(0.1)
    ledpattern(0, 0, 0, 0, 0, 0, 1, 0)
    time.sleep(0.1)
    ledpattern(0, 0, 0, 0, 0, 1, 0, 0)
    time.sleep(0.1)
    ledpattern(0, 0, 0, 0, 1, 0, 0, 0)
    time.sleep(0.1)
    ledpattern(0, 0, 0, 1, 0, 0, 0, 0)
    time.sleep(0.1)
    ledpattern(0, 0, 1, 0, 0, 0, 0, 0)
    time.sleep(0.1)
    ledpattern(0, 1, 0, 0, 0, 0, 0, 0)
    time.sleep(0.1)
    ledpattern(1, 0, 0, 0, 0, 0, 0, 0)
    time.sleep(0.1)
```

EDKITS

```
def patternFour():
  for i in range (0, 5):
    ledpattern(0, 1, 1, 1, 1, 1, 1, 1)
    time.sleep(0.1)
    ledpattern(1, 0, 1, 1, 1, 1, 1, 1)
    time.sleep(0.1)
    ledpattern(1, 1, 0, 1, 1, 1, 1, 1)
    time.sleep(0.1)
    ledpattern(1, 1, 1, 0, 1, 1, 1, 1)
    time.sleep(0.1)
    ledpattern(1, 1, 1, 1, 0, 1, 1, 1)
    time.sleep(0.1)
    ledpattern(1, 1, 1, 1, 1, 0, 1, 1)
    time.sleep(0.1)
    ledpattern(1, 1, 1, 1, 1, 1, 0, 1)
    time.sleep(0.1)
    ledpattern(1, 1, 1, 1, 1, 1, 1, 0)
    time.sleep(0.1)

def patternFive():
  for i in range (0, 5):
    ledpattern(1, 1, 1, 1, 1, 1, 1, 0)
    time.sleep(0.1)
    ledpattern(1, 1, 1, 1, 1, 1, 0, 1)
    time.sleep(0.1)
    ledpattern(1, 1, 1, 1, 1, 0, 1, 1)
    time.sleep(0.1)
    ledpattern(1, 1, 1, 1, 0, 1, 1, 1)
    time.sleep(0.1)
    ledpattern(1, 1, 1, 0, 1, 1, 1, 1)
    time.sleep(0.1)
    ledpattern(1, 1, 0, 1, 1, 1, 1, 1)
    time.sleep(0.1)
    ledpattern(1, 0, 1, 1, 1, 1, 1, 1)
    time.sleep(0.1)
    ledpattern(0, 1, 1, 1, 1, 1, 1, 1)
    time.sleep(0.1)

try:
  while True:
    patterOne()
    patternTwo()
    patternThree()
    patternFour()
    patternFive()

finally:
  #reset the GPIO Pins
  GPIO.cleanup()
```

EDKITS

Displaying Time over 4 Digit 7 Segment Display using Raspberry Pi

In this lesson we will interface 4-digit seven segment display with raspberry pi.

Hardware Guide:

For completing this lesson, you will require the following things along with your initial raspberry pi setup

1. TM1637 4-digit seven segment Display board
2. Connecting wires

TM1637 4 Digit seven segment Display Board:

This is a common anode 4-digit tube display module which uses the TM1637 driver chip; Only 2 connections are required to control the 4-digit 8-segment displays

Here is the module



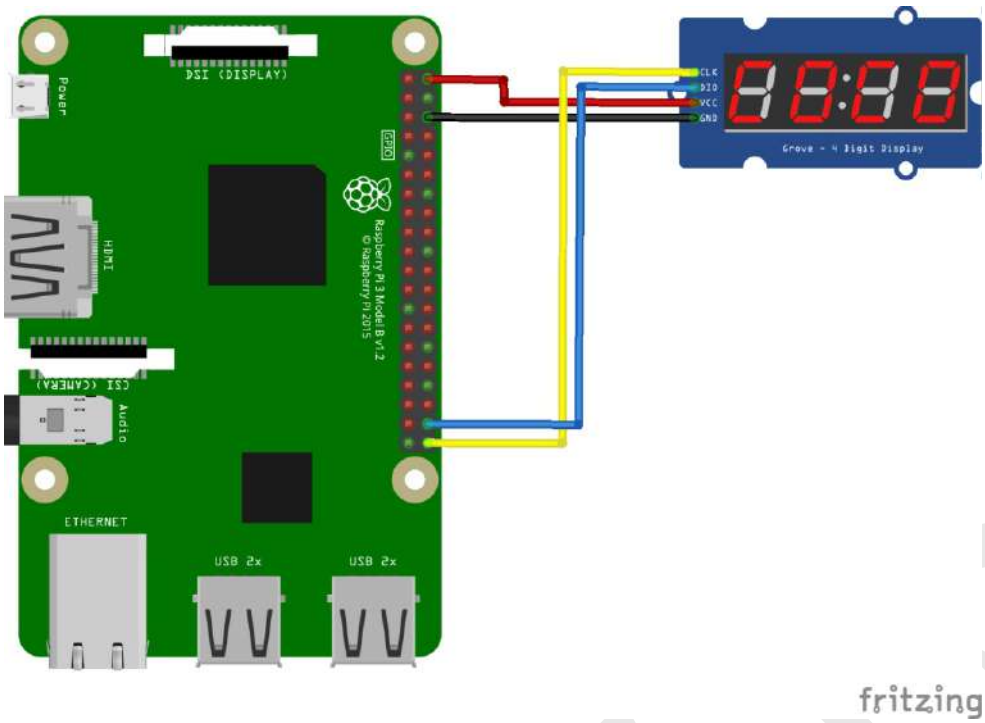
Features of the module:

1. Display common anode for the four red LED
2. Powered supply by 3.3V/5V
3. Four common anode tube display module is driven by IC TM1637

Wiring up your Circuit:

Hook up your circuit as follows:

1. Connect the Pin2 (5V) of Rpi to Vcc pin of Module
2. Connect Pin 6 (GND) of Rpi to GND of Module
3. Connect Pin38 (GPIO20) of Rpi to DIO of Module
4. Lastly connect Pin 40 (GPIO21) of Rpi to CLK of Module



Software Guide:

1. Now to download libraries, open Web Browser on your Raspberry Pi and log on to the following link: **https://github.com/timwaizenegger/raspberrypi-examples/tree/master/actor-led-7segment-4numbers** . Click on the actor-led-7segment-4numbers.zip folder and Now click on Download Button to download the file.
2. Now on your rpi move to /home/pi/Downloads/ location to find the zip file downloaded.
3. Unzip the file and try to execute the different example codes present in that folder in Python 2 Idle.
4. Now open Python 2 Idle, create a new file, write the code given below and save it in the same folder i.e. actor-led-7segment-4numbers since the code below is depended on tm1637.py file which is present in the same folder.

Code:

```
#Program to display Time on 4-digit Seven segment display
from time import sleep
import tm1637

try:
    import thread
except ImportError:
    import _thread as thread

# Initialize the clock (GND, VCC=3.3V, Example Pins are DIO -20 and CLK21)
```

```
Display = tm1637.TM1637(CLK=21, DIO=20, brightness=1.0)

try:
    print ("Starting clock in the background (press CTRL + C to stop):")
    Display.StartClock(military_time=True)
    Display.SetBrightness(1.0)
    while True:
        Display.ShowDoublepoint(True)
        sleep(1)
        Display.ShowDoublepoint(False)
        sleep(1)

    Display.StopClock()
    thread.interrupt_main()
except KeyboardInterrupt:
    print ("Properly closing the clock and open GPIO pins")
    Display.cleanup()
```

(Note: the code given above is the edited form of clock.py program in 'actor-led-7segment-4number' folder.)

EDKITS

Interfacing 16x2 LCD (I2C) Display with Raspberry Pi

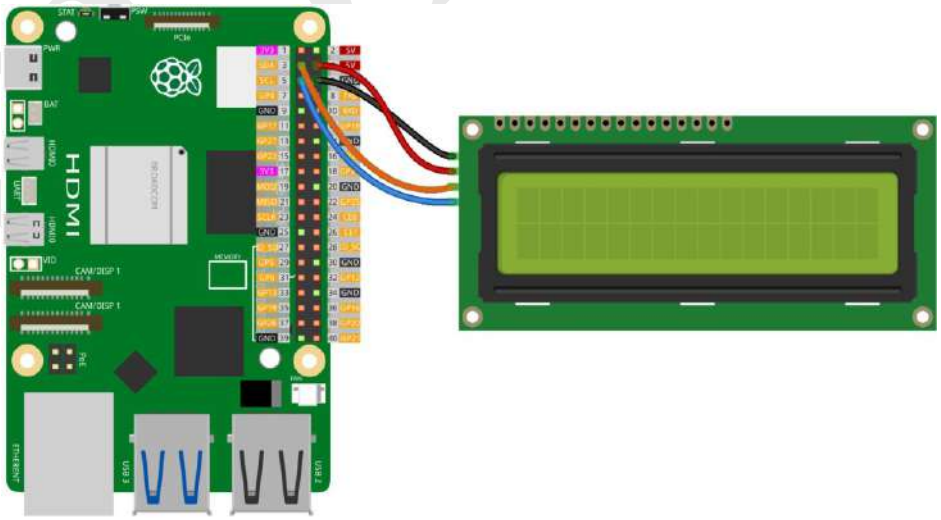
Building the 16×2 LCD (I2C) Display Circuit:

1. To start our tutorial, we will first begin setting up the 16×2 LCD with I2C Module.
2. Connect I2C Module to LCD as shown in the given image below.



Wiring Details:

To connect a 16×2 LCD with an I2C backpack to a Raspberry Pi, you only need four wires.



WIRING I2C LCD	Raspberry Pi GPIO
VCC	5V (Pin 2 or Pin 4)
GND	GND (Pin 6)
SDA	GPIO 2 (Pin 3)
SCL	GPIO 3 (Pin 5)

Software Guide:

Install Dependencies for I2C LCD:

Before we start writing the python script, we need to enable the I2C communication interface of the raspberry pi and install the software requirements that were mentioned earlier. This will be done in below steps so its easy to follow:

Step 1: Enable Raspberry Pi I2C interface

sudo raspi-config

When the configuration panels open, select interface options, select I2C and click enable.

Then reboot

sudo reboot

Step 2: Update the Raspberry Pi

The first thing I do before starting any project is updating the Pi. Through this, I am sure every thing on the OS is up to date and I won't experience compatibility issue with any latest software I choose to install on the Pi. To do this, run below two commands:

sudo apt-get update

Step 3: Install required Packages

sudo apt install -y python3-smbus i2c-tools

sudo pip3 install RPLCD smbus2

pip3 install RPLCD

Step 4: Check the I2C address

i2cdetect -y 1

You should see an address such as:

27

or

3f

Use that address in your code.

EDKITS

PYTHON CODE

```
from RPLCD.i2c import CharLCD
from time import sleep

lcd = CharLCD('PCF8574', 0x27)

lcd.write_string("Hello World!")
sleep(5)

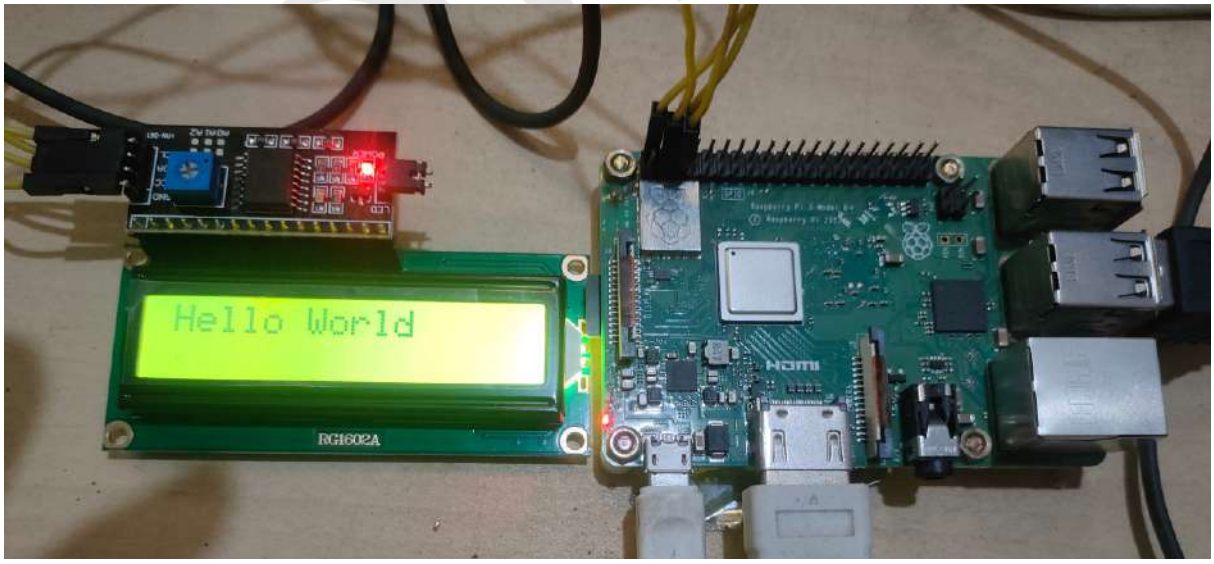
#Switch off backlight
lcd.backlight_enabled = False

#Clear
lcd.close(clear=True)
```

Step 5

To test that everything is working lets now run that python script by running the command below. If everything is working as it should, you should now see text displayed across your LCD.

pi@raspberrypi:~ $ **sudo python3 lcd.py**



EDKITS

Oscilloscope Using Raspberry Pi

The oscilloscope is an electronic test instrument that allows the visualization and observation of varying signal voltages, usually as a two dimensional plot with one or more signals plotted against time. Today's project will seek to replicate the signal visualization capabilities of the oscilloscope using the Raspberry Pi and an analog to digital converter module.

Replicating the signal visualization of the oscilloscope using the Raspberry Pi will require the following steps;

1. Perform Digital to analog conversion of the Input signal
2. Prepare the resulting data for representation
3. Plot the data on a live time graph

Hardware requirements:

To build this project, the following components/part are required;

1. 1.Raspberry pi 4 (or any other model)
2. 8 or 16GB SD Card
3. LAN/Ethernet Cable
4. Power Supply or USB cable
5. ADS1115 ADC
6. 10k or 1k resistor
7. Jumper wires
8. Breadboard
9. Monitor or any other way of seeing the pi's Desktop(VNC inclusive)

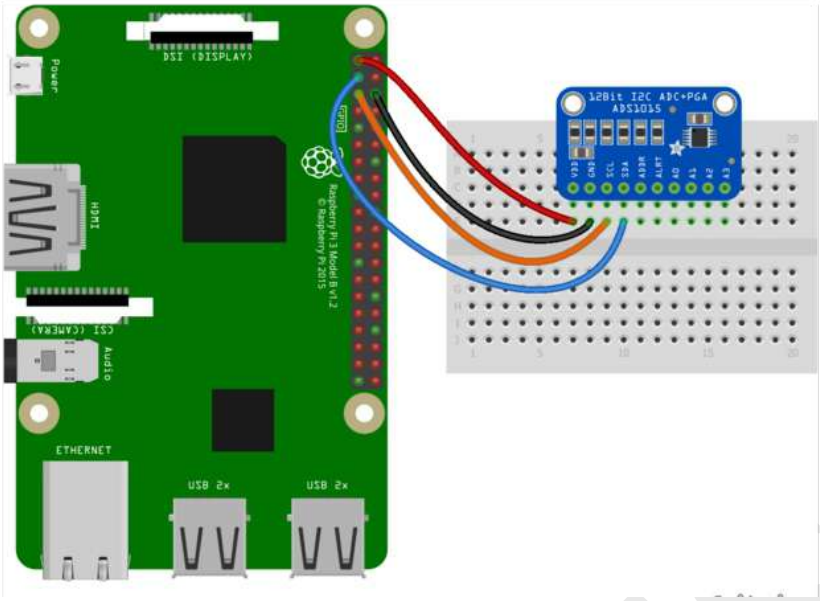
Wiring up your Circuit:

To convert the analog input signals to digital signals which can be visualized with the Raspberry Pi, we will be using the ADS1115 ADC chip. This chip becomes important because the Raspberry Pi, unlike Arduino and most micro-controllers, does not have an on-board analog to digital converter(ADC).

The ADC is an I2C based device and should be connected to the Raspberry Pi as shown in the schematics below.

For clarity, the pin connection between the two components is also described below.

EDKITS



ADS1115 and Raspberry Pi Connections:

1. **VDD – 3.3v Raspberry pi pin 1**
2. **GND – GND Raspberry pi pin 6**
3. **SCL – SCL Raspberry pi pin 5**
4. **SDA – SDA Raspberry pi pin 3**

With the connections all done, power up your pi and proceed to install the dependencies mentioned below.

Software Guide:

Install Dependencies for Raspberry Pi Oscilloscope:

Before we start writing the python script to pull data from the ADC and plot it on a live graph, we need to enable the I2C communication interface of the raspberry pi and install the software requirements that were mentioned earlier. This will be done in below steps so its easy to follow:

Step 1: Enable Raspberry Pi I2C interface

sudo raspi-config

When the configuration panels open, select interface options, select I2C and click enable.

Step 2: Update the Raspberry Pi

The first thing I do before starting any project is updating the Pi. Through this, I am sure every thing on the OS is up to date and I won't experience compatibility issue with any latest software I choose to install on the Pi. To do this, run below two commands:

sudo apt-get update

EDKITS

Step 3: Install the Adafruit ADS1115 library for ADC

With the update done, we are now ready to install the dependencies starting with the Adafruit python module for the ADS115 chip. Ensure you are in the Raspberry Pi home directory by running;

cd ~

then install the build-essentials by running;

sudo apt-get install build-essential python-dev python-smbus git

Next, clone the Adafruit git folder for the library by running;

git clone https://github.com/adafruit/Adafruit_Python_ADS1x15.git

Change into the cloned file's directory and run the setup file;

cd Adafruit_Python_ADS1x15

sudo python3 setup.py install

Step 4: Test the library and 12C communication.

Before we proceed with the rest of the project, it is important to test the library and ensure the ADC can communicate with the raspberry pi over I2C. To do this we will use an example script that comes with the library.

While still in the Adafruit_Python_ADS1x15 folder, change directory to the examples directory by running;

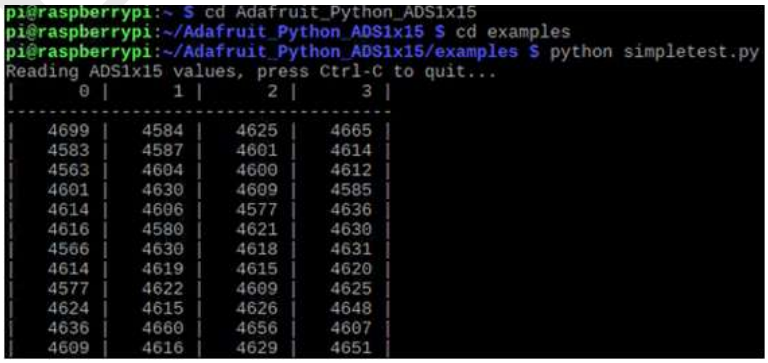
cd examples

Next, run the sampletest.py example which displays the value of the four channels on the ADC in a tabular form.

Run the example using:

python3 simpletest.py

If the I2C module is enabled and connections good, you should see the data as shown in the image below.



If an error occurs, check to ensure the ADC is well connected to the PI and I2C communication is enabled on the Pi.

EDKITS

Step 5: Install Matplotlib

To visualize the data we need to install the matplotlib module which is used to plot all kind of graphs in python. This can be done by running;

sudo apt-get install python3-matplotlib

Step 6: Install the Drawnow python module

Lastly, we need to install the drawnow python module. This module helps us provide live updates to the data plot.

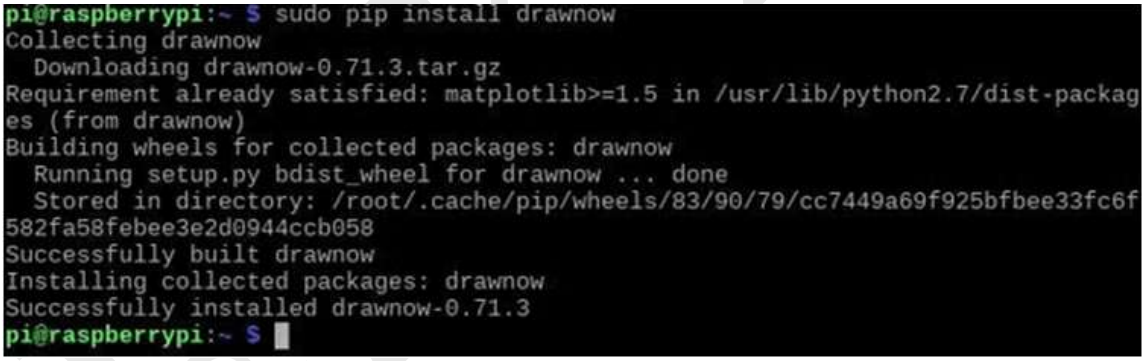
We will be installing drawnow via the python package installer; pip, so we need to ensure it is installed. This can be done by running;

sudo apt-get install python3-pip

We can then use pip to install the drawnow package by running:

sudo pip3 install drawnow

You should get an outcome like the image below after running it.



With all the dependencies installed, we are now ready to write the code.

Python Code for Raspberry Pi Oscilloscope:

The python code for this Pi Oscilloscope is fairly simple especially if you are familiar with the python matplotlibmodule. Before showing us the whole code, I will try to break it into part and explain what each part of the code is doing so you can have enough knowledge to extend the code to do more stuffs.

At this stage it is important to switch to a monitor or use the VNC viewer, anything through which you can see your Raspberry Pi's desktop, as the graph being plotted won't show on the terminal.

With the monitor as the interface open a new python file. You can call it any name you want, but I will call it scope.py.

sudo nano scope.py

Code

```
import time
import matplotlib.pyplot as plt
#import numpy
from drawnow import *
# Import the ADS1x15 module.
import Adafruit_ADS1x15
# Create an ADS1115 ADC (16-bit) instance.
adc = Adafruit_ADS1x15.ADS1115()

GAIN = 1
val = [ ]
cnt = 0
plt.ion()
# Start continuous ADC conversions on channel 0 using the previous gain value.
adc.start_adc(0, gain=GAIN)
print('Reading ADS1x15 channel 0')
#create the figure function
def makeFig():
  plt.ylim(-5000,5000)
  plt.title('Osciloscope')
  plt.grid(True)
  plt.ylabel('ADC outputs')
  plt.plot(val, 'ro-', label='Channel 0')
  plt.legend(loc='lower right')
while (True):
  # Read the last ADC conversion value and print it out.
  value = adc.get_last_result()
  print('Channel 0: {0}'.format(value))
  # Sleep for half a second.
  time.sleep(0.5)
  val.append(int(value))
  drawnow(makeFig)
  plt.pause(.000001)
  cnt = cnt+1
  if(cnt>50):
    val.pop(0)
```

EDKITS

Raspberry Pi Oscilloscope in Action:

Copy the complete python code and paste in the python file we created earlier, remember we will need a monitor to view the plot so all of this should be done by either VNC or with a connected monitor or screen.

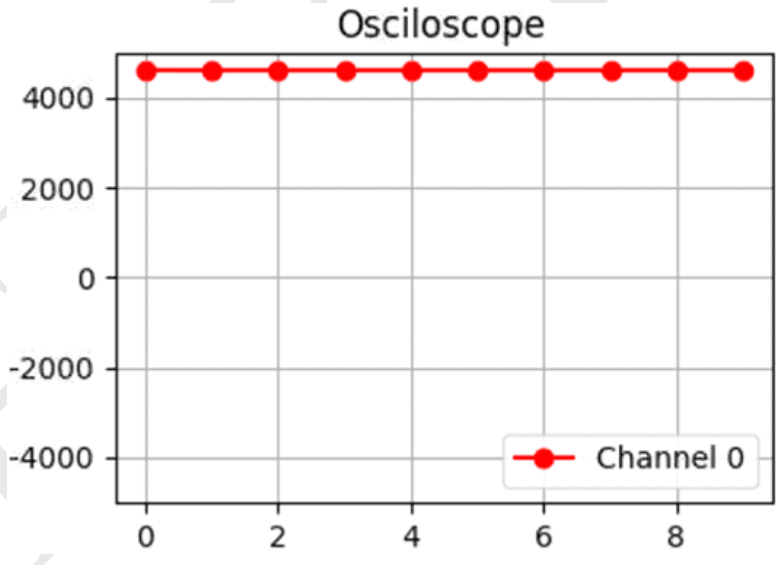
Save the code and run using;

sudo python3 scope.py

If you used a different name other than scope.py, don't forget to change this to match.

After a few minutes, you should see the ADC data being printed on the terminal.

```
pi@raspberrypi:~ $ sudo nano scope.py
pi@raspberrypi:~ $ sudo python scope.py
Reading ADS1x15 channel 0
Channel 0: 4618
/usr/lib/python2.7/dist-packages/matplotlib/backend
plotlibDeprecationWarning: Using default event loop
cific to this GUI is implemented
  warnings.warn(str, mplDeprecation)
Channel 0: 4615
Channel 0: 4616
Channel 0: 4615
Channel 0: 4614
Channel 0: 4613
Channel 0: 4614
```



You can connect an analog device like a potentiometer to a channel on the ADC and you should see the data change with each turn of the potentiometer. Or you can input Sine wave or square wave to test the output.

NOTE: You can find the entire details at https://circuitdigest.com/microcontroller-projects/raspberry-pi-based-oscilloscope

Interfacing Raspberry Pi with Telegram App

On 24 June, 2015, Telegram published the Bot API, enabling machines to talk Telegram. From that day on, not only can human use Telegram, so can machines.

For those who don't know what Telegram is, it is a messaging app, very much like WhatsApp. This tutorial is going to teach how to send a Telegram message to your Raspberry Pi, and how to make your Pi telegram back. Make sure the Pi has internet access.

Step 1: Install Telegram on Your Phone, Obviously

Go to App Store (iPhone) or Play Store (Android), download and install Telegram on your phone.

Now, you can use Telegram. Not yet for the Raspberry Pi. Telegram reserves a special kind of accounts for machines, called bot accounts. As the owner of your own Pi, you have to obtain a bot account for it.

Step 2: Text /newbot to BotFather

Open Telegram on your phone, search for a user called BotFather. As the name implies, he is the Father of All Bots.

As you may have guessed, he is not of our own species, but is actually a machine. He accepts special commands, because he does not understand plain English very well.

To obtain a bot account, text him /newbot. (you need the slash '/' in front) He will then ask a couple of questions. I call my bot "Led". You will see why in a few moments. But you can give it any name you want.

At the end of process, you will be given a token, something like 123456789:ABCdefGhIJKlmNo-PQRsTUVwxyZ. This token represents the bot account. You are going to put this token on the Pi.

Step 3: Install Telepot on Raspberry Pi

Enter the Pi, via SSH or a USB-TTL serial cable. Install telepot, a Python package that enables the Pi to speak Telegram Bot API.

On the command line, run these two commands:

```
sudo apt-get install python-pip
sudo pip install telepot
```

Step 4: Test Token

On the command line, type python to enter the Python interpreter.

In the Python interpreter, enter these three lines;

```
import telepot
bot = telepot.Bot('*** copy bot token from botfather ***')
bot.getMe()
```

You should keep bot token secret too. Having the token means having access to the bot account.

If the last command, getMe(), returns a dictionary describing the bot account (as in the screenshot), all is good. Type exit() to leave the Python interpreter.

If not, you have copied the wrong token. Type exit() to leave the Python interpreter. Then type python to come in again, and repeat those three lines of code.

EDKITS

Step 5: Python code?

Create a python file name telegram.py in raspberry pi and copy the following code.

```
import sys
import time
import random
import datetime
import telepot
import RPi.GPIO as GPIO
from telepot.loop import MessageLoop

red=40                  # connect red led at pin 40 of raspberry pi

now=datetime.datetime.now()
GPIO.setmode(GPIO.BOARD)
GPIO.setwarnings(False)

GPIO.setup(red,GPIO.OUT)
GPIO.output(red,0)

def action(msg):
   chat_id = msg['chat']['id']
   command = msg['text']

   print ('Got command: %s' %command)
   if 'On' in command:
      message="Turn On"
      message=message+" red"
      GPIO.output(red,1)
      bot.sendMessage(chat_id,message)

   if 'Off' in command:
      message="Turn Off"
      message=message+" red"
      GPIO.output(red,0)
      bot.sendMessage(chat_id, message)

bot = telepot.Bot('626665131:AAHsNzQbqSj9GZ9-w2t4I')        #paste your bot tokan here.
print(bot.getMe())
MessageLoop(bot,action).run_as_thread()
print ('I am listening...')

while 1:
    time.sleep(10)
```

Step 6: Connect red LED at pin 40 (GPIO21)

Step 7: Run It and Text It

Assuming you have named the file you have just saved "telegram.py", to run the bot, type

```
sudo python /home/pi/telegram.py
```

Open Telegram on your phone, search for your bot using its name or username. Text it On or Off, and see how it responds.

EDKITS

Fingerprint Sensor interfacing with Raspberry Pi with onboard UART

In this lesson we will learn to interface fingerprint sensor with raspberry pi. For this we willing be using the USB port of raspberry pi.

Hardware Guide:

For completing this lesson, you will require the following things along with your initial raspberry pi setup

1. Fingerprint Sensor
2. Connecting wires
3. Bread Board

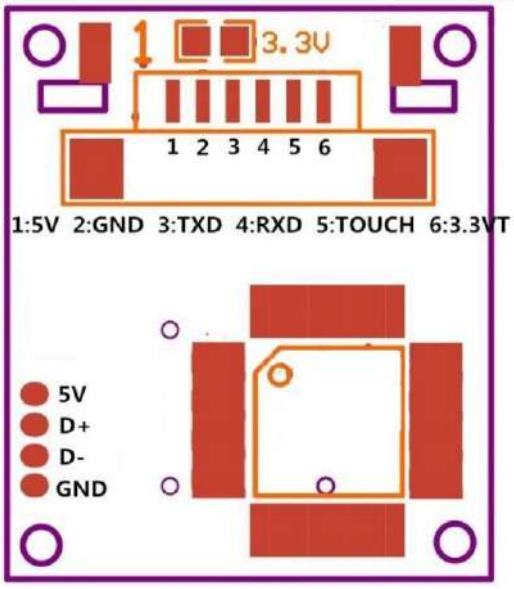
Fingerprint Sensor:

It is an intelligent module which can freely get fingerprint, image processing, verified fingerprint, search and storage, and it can work normally without upper monitor's participatory management.

Fingerprint processing includes two parts: fingerprint enrolment and fingerprint matching (the matching can be 1:1 or 1: N). Enrolling fingerprint, user needs to enter the finger 2-4 times for every one finger, process finger images with many times, store generate templates on module. When fingerprint matching, enrol and process verified fingerprint image and then matching with

PIn 1 = +5V, Pin2 = GND, PIN 3 = TXD , Pin 4 = RXD





EDKITS ELECTRONICS

module (if match with appoint templates on the module, named fingerprint verification, for 1:1 matching method; if match with many templates on the module, named fingerprint search method also named 1: N) system will return the matching result, success or failure.

Enabling Built in UART

You'll need to install the Adafruit_Blinka library that provides the CircuitPython support in Python. This may also require enabling the hardware UART on your Raspberry Pi.

As we are using the built-in UART, you'll need to disable the serial console and enable the serial port hardware in raspi-config. Follow the given procedure to enable built in UART.

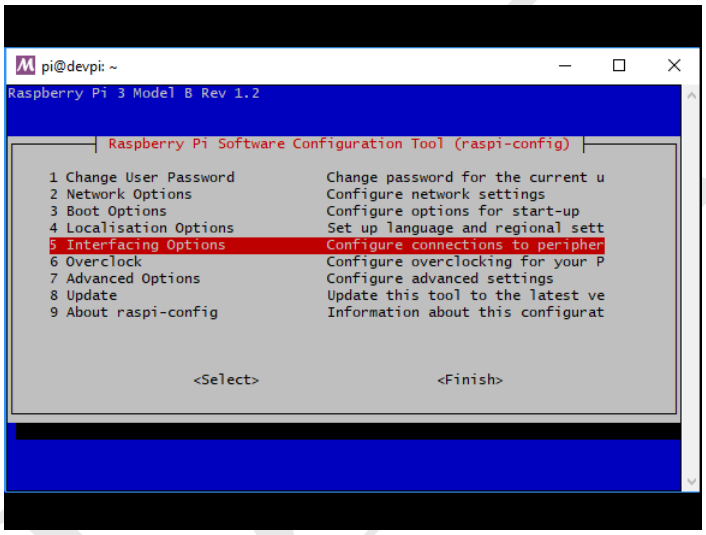
Disabling Console & Enabling Serial

Before wiring up, make sure you have disabled the console. Run the following command in console.

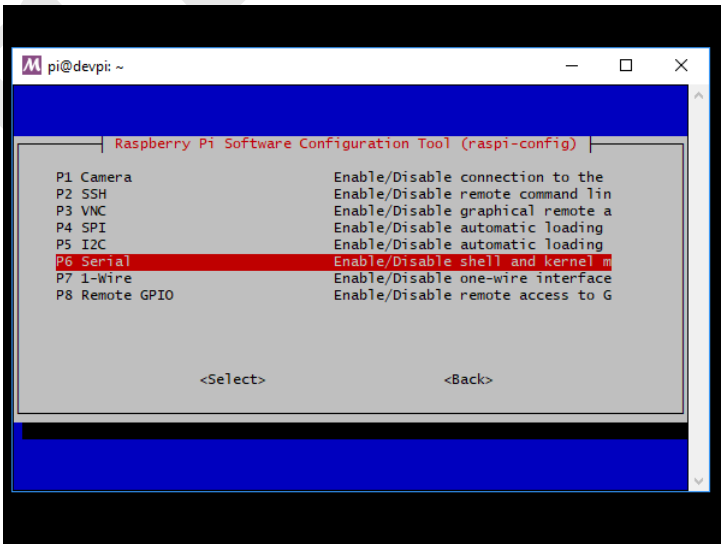
sudo raspi-config

and select the following:

Interfacing Options



Serial



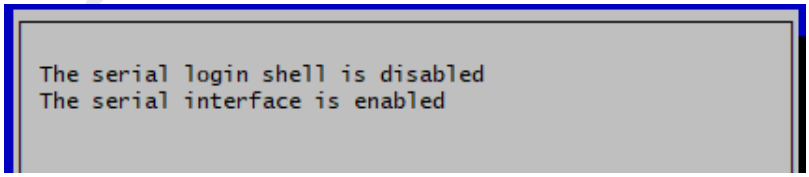
Select No on enabling the login shell



Select Yes on enabling serial port hardware



Once complete you should have no console and yes on serial interface:



Then **reboot**

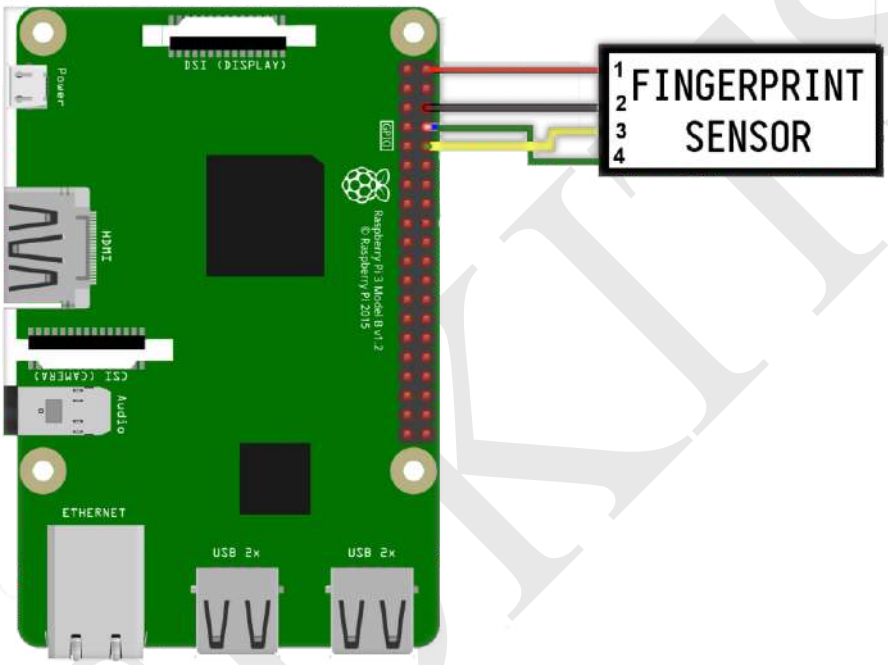
Once you've rebooted, you can use the built in **UART via /dev/ttyS0**

Wiring up your circuit:

In this Raspberry Pi Finger Print sensor interfacing project we connect Finger Print Sensor to Raspberry Pi UART.

So, first of all, we need to make the all the required connection as shown below.
Connections are simple, we have just connected fingerprint module to Raspberry Pi UART port

1. Connect the VCC Pin of Finger Print Module (Pin 1) Red Wire to 5V Pin Pin 2 of Raspberry Pi.
2. Connect the GND Pin of Finger Print Module (Pin 2) Black Wire to GND Pin 6 of Raspberry Pi.
3. Connect the TXD Pin of Finger Print Module (Pin 3) Yellow Wire to RXD Pin 10 of Raspberry Pi.
4. Connect the RXD Pin of Finger Print Module (Pin 4) Green Wire to TXD Pin 8 of Raspberry Pi.



Software Guide:

Python Installation of Fingerprint Library

After making all the connections we need to power up Raspberry Pi and get it ready with terminal open. Now we need to install fingerprint library for Raspberry Pi in python language by following the below steps. From your command line run the following command:

sudo pip3 install adafruit-circuitpython-fingerprint

it will install adafruit circuitpython fingerprint

CODE :

```
import time
import board
from digitalio import DigitalInOut, Direction
import adafruit_fingerprint

led = DigitalInOut(board.D13)
led.direction = Direction.OUTPUT

# If using with Linux/Raspberry Pi and hardware UART:
import serial
uart = serial.Serial("/dev/ttyS0", baudrate=57600, timeout=1)

finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)

##################################################


def get_fingerprint():
    """Get a finger print image, template it, and see if it matches!"""
    print("Waiting for image...")
    while finger.get_image() != adafruit_fingerprint.OK:
        pass
    print("Templating...")
    if finger.image_2_tz(1) != adafruit_fingerprint.OK:
        return False
    print("Searching...")
    if finger.finger_search() != adafruit_fingerprint.OK:
        return False
    return True

# pylint: disable=too-many-branches
def get_fingerprint_detail():
    """Get a finger print image, template it, and see if it matches!
    This time, print out each error instead of just returning on failure"""
    print("Getting image...", end="")
    i = finger.get_image()
    if i == adafruit_fingerprint.OK:
        print("Image taken")
    else:
        if i == adafruit_fingerprint.NOFINGER:
            print("No finger detected")
        elif i == adafruit_fingerprint.IMAGEFAIL:
            print("Imaging error")
        else:
            print("Other error")
        return False
```

EDKITS

```
    print("Templating...", end="")
    i = finger.image_2_tz(1)
    if i == adafruit_fingerprint.OK:
        print("Templated")
    else:
        if i == adafruit_fingerprint.IMAGEMESS:
            print("Image too messy")
        elif i == adafruit_fingerprint.FEATUREFAIL:
            print("Could not identify features")
        elif i == adafruit_fingerprint.INVALIDIMAGE:
            print("Image invalid")
        else:
            print("Other error")
        return False

    print("Searching...", end="")
    i = finger.finger_fast_search()
    # pylint: disable=no-else-return
    # This block needs to be refactored when it can be tested.
    if i == adafruit_fingerprint.OK:
        print("Found fingerprint!")
        return True
    else:
        if i == adafruit_fingerprint.NOTFOUND:
            print("No match found")
        else:
            print("Other error")
        return False


# pylint: disable=too-many-statements
def enroll_finger(location):
    """Take a 2 finger images and template it, then store in 'location'"""
    for fingerimg in range(1, 3):
        if fingerimg == 1:
            print("Place finger on sensor...", end="")
        else:
            print("Place same finger again...", end="")

        while True:
            i = finger.get_image()
            if i == adafruit_fingerprint.OK:
                print("Image taken")
                break
            if i == adafruit_fingerprint.NOFINGER:
                print(".", end="")
            elif i == adafruit_fingerprint.IMAGEFAIL:
                print("Imaging error")
                return False
            else:
                print("Other error")
                return False
```

EDKITS

```
        print("Templating...", end="")
        i = finger.image_2_tz(fingerimg)
        if i == adafruit_fingerprint.OK:
            print("Templated")
        else:
            if i == adafruit_fingerprint.IMAGEMESS:
                print("Image too messy")
            elif i == adafruit_fingerprint.FEATUREFAIL:
                print("Could not identify features")
            elif i == adafruit_fingerprint.INVALIDIMAGE:
                print("Image invalid")
            else:
                print("Other error")
            return False

        if fingerimg == 1:
            print("Remove finger")
            time.sleep(1)
            while i != adafruit_fingerprint.NOFINGER:
                i = finger.get_image()

    print("Creating model...", end="")
    i = finger.create_model()
    if i == adafruit_fingerprint.OK:
        print("Created")
    else:
        if i == adafruit_fingerprint.ENROLLMISMATCH:
            print("Prints did not match")
        else:
            print("Other error")
        return False

    print("Storing model #%d..." % location, end="")
    i = finger.store_model(location)
    if i == adafruit_fingerprint.OK:
        print("Stored")
    else:
        if i == adafruit_fingerprint.BADLOCATION:
            print("Bad storage location")
        elif i == adafruit_fingerprint.FLASHERR:
            print("Flash storage error")
        else:
            print("Other error")
        return False

  return True

##################################################

def get_num():
    """Use input() to get a valid number from 1 to 127. Retry till success!"""
    i = 0
    while (i > 127) or (i < 1):
        try:
            i = int(input("Enter ID # from 1-127: "))
        except ValueError:
            pass
    return i
```

EDKITS

```
while True:
    print("----------------")
    if finger.read_templates() != adafruit_fingerprint.OK:
        raise RuntimeError("Failed to read templates")
    print("Fingerprint templates:", finger.templates)
    print("e) enroll print")
    print("f) find print")
    print("d) delete print")
    print("----------------")
    c = input("> ")

    if c == "e":
        enroll_finger(get_num())
    if c == "f":
        if get_fingerprint():
            print("Detected #", finger.finger_id, "with confidence", finger.confidence)
        else:
            print("Finger not found")
    if c == "d":
        if finger.delete_model(get_num()) == adafruit_fingerprint.OK:
            print("Deleted!")
        else:
            print("Failed to delete")
```

It's fairly long but it will help you set-up and test your sensor. Now Run

Fingprint1.py

When you first start up, you should get something like this:

```
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
main.py output:
----------------
Fingerprint templates: [2]
e) enroll print
f) find print
d) delete print
----------------
>
```

If you get an error like RuntimeError: Failed to read data from sensor it means something went wrong - check your wiring and baud rate!

This menu system is fairly simple, you have three things you can do

Enroll print **e** - you will use your finger to take images and 'store' the model in the sensor
Find print **f** - determine whether a fingerprint is known and stored
Delete print **d** - clear out a model

EDKITS

Fingerprint Sensor interfacing with Raspberry Pi using USB to TTL Converter

In this lesson we will learn to interface fingerprint sensor with raspberry pi. For this we willing be using the USB port of raspberry pi.

Hardware Guide:

For completing this lesson, you will require the following things along with your initial raspberry pi setup

1. Fingerprint Sensor
2. USB to TTL/UART converter
3. Connecting wires
4. Push Buttons
5. LED
6. Breadboard

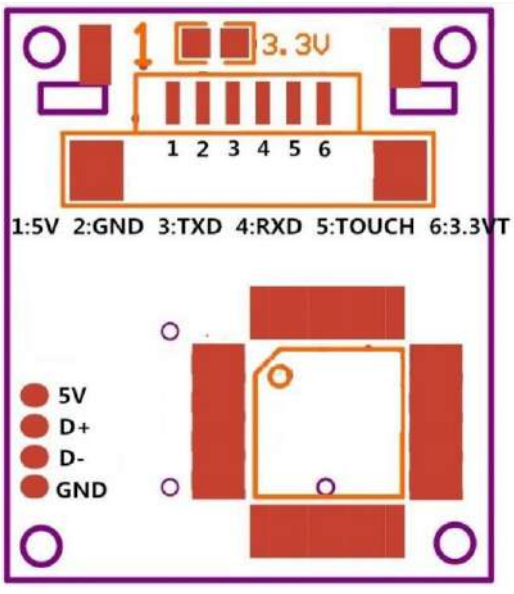
Fingerprint Sensor:

It is an intelligent module which can freely get fingerprint, image processing, verified fingerprint, search and storage, and it can work normally without upper monitor's participatory management.

Fingerprint processing includes two parts: fingerprint enrolment and fingerprint matching (the matching can be 1:1 or 1: N). Enrolling fingerprint, user needs to enter the finger 2-4 times for every one finger, process finger images with many times, store generate templates on module. When fingerprint matching, enrol and process verified fingerprint image and then matching with

PIn 1 = +5V, Pin2 = GND, PIN 3 = TXD , Pin 4 = RXD



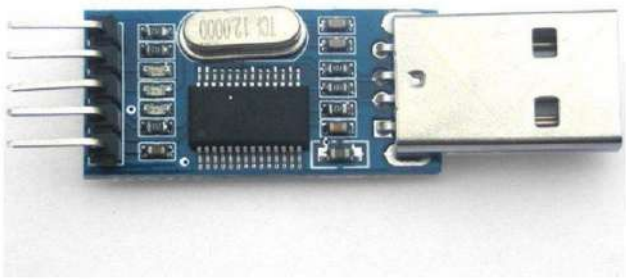


EDKITS

module (if match with appoint templates on the module, named fingerprint verification, for 1:1 matching method; if match with many templates on the module, named fingerprint search method also named 1: N) system will return the matching result, success or failure.

USB to TTL converter:

Here we have used a fingerprint module which works on UART. So here we have interfaced this fingerprint module with Raspberry Pi using a USB to Serial converter.

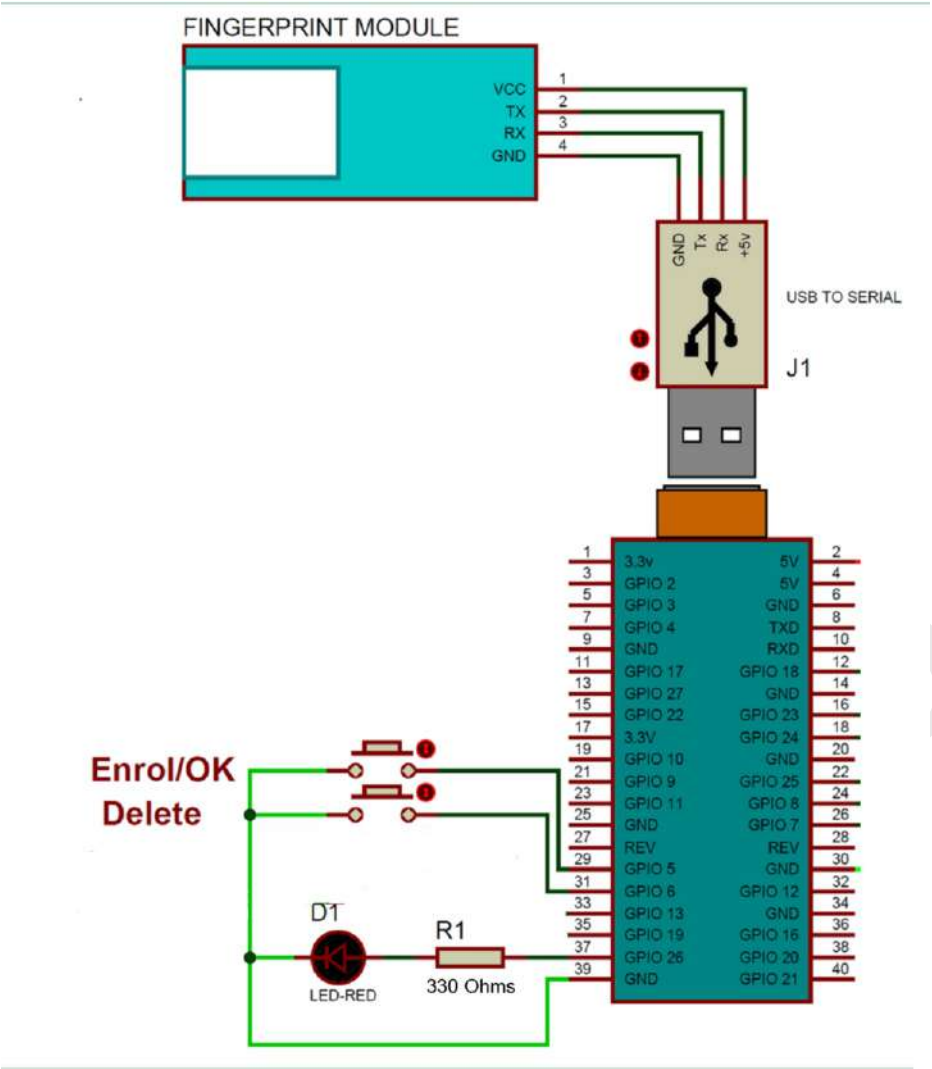


Wiring up your circuit:

In this Raspberry Pi Finger Print sensor interfacing project, we have used a 2 push buttons: one for enrolling the new finger print, one for deleting the already fed finger prints positions.
A LED is used for indication that fingerprint sensor is ready to take finger for matching. Here we have used a fingerprint module which works on UART. So here we have interfaced this fingerprint module with Raspberry Pi using a USB to Serial converter.

So, first of all, we need to make the all the required connection as shown below.
Connections are simple, we have just connected fingerprint module to Raspberry Pi USB port by using USB to Serial converter.

1. Connect the VCC Pin of Finger Print Module (Pin 1) Red Wire to 5V Pin of USB to TTL converter
2. Connect the GND Pin of Finger Print Module (Pin 2) Black Wire to GND Pin of USB to TTL converter
3. Connect the TXD Pin of Finger Print Module (Pin 3) Yellow Wire to RXD Pin of USB to TTL converter
4. Connect the RXD Pin of Finger Print Module (Pin 4) Green Wire to TXD Pin of USB to TTL converter
5. Lastly connect the USB to TTL converter to USB port of Raspberry Pi.
6. Connect one push button switch to pin 29 of Raspberry Pi for Enroll finger print.
7. Connect another push button switch to pin 31 of Raspberry Pi for Deleting already fed finger print.
8. Connect LED to pin 37 of Raspberry Pi for indication that finger print is ready to take finger for matching.
9. Finally connect ground pin 39(GND) of Raspberry pi to circuit as shown in Circuit diagram.



Software Guide:

After making all the connections we need to power up Raspberry Pi and get it ready with terminal open. Now we need to install fingerprint library for Raspberry Pi in python language by following the below steps.

1. To install this library, root privileges are required. So first we enter in root by given command: sudo bash
2. Then download some required packages by using given commands:
    a. wget –O – http://apt.pm-codeworks.de/pm-codeworks.de.gpg | apt-key add –
    b. wget http://apt.pm-codeworks.de/pm-codeworks.list -P /etc/apt/sources.list.d/
3. After this, we need to update the Raspberry pi and install the downloaded finger print sensor library:
    a. sudo apt-get update
    b. sudo apt-get install python-fingerprint
    c. now exit root by typing exit
4. After installing library now, we need to check USB port on which your finger print sensor is connected, by using given the command: ls /dev/ttyUSB* (or lsusb)

Now in your python code replace the USB port number with the one you got on the screen after executing the command in step4.

Code:

```
import time
from pyfingerprint.pyfingerprint import PyFingerprint
import RPi.GPIO as gpio


enrol=5
delet=6
led=26

HIGH=1
LOW=0

gpio.setwarnings(False)
gpio.setmode(gpio.BCM)

gpio.setup(enrol, gpio.IN, pull_up_down=gpio.PUD_UP)
gpio.setup(delet, gpio.IN, pull_up_down=gpio.PUD_UP)
gpio.setup(led, gpio.OUT)

try:
    f = PyFingerprint('/dev/ttyUSB0', 57600, 0xFFFFFFFF, 0x00000000)
    if ( f.verifyPassword() == False ):
        raise ValueError('The given fingerprint sensor password is wrong!')
except Exception as e:
    print('Exception message: ' + str(e))
    exit(1)

print('Currently used finger templates: ' + str(f.getTemplateCount()) +'/'+ str(f.getStorageCapacity()))


def enrollFinger():
    print('Waiting for finger...')
    while ( f.readImage() == False ):
        pass
    f.convertImage(0x01)
    result = f.searchTemplate()
    positionNumber = result[0]
    if ( positionNumber >= 0 ):
        print('Finger already exists at position #' + str(positionNumber))
        time.sleep(2)
        return
    print('Remove finger...')
    time.sleep(2)


    print('Waiting for same finger again...')
    while ( f.readImage() == False ):
        pass
    f.convertImage(0x02)
    if ( f.compareCharacteristics() == 0 ):
        print ("Fingers do not match")
        time.sleep(2)
        return
```

EDKITS

```
    f.createTemplate()
    positionNumber = f.storeTemplate()
    print('Finger enrolled successfully!')

    print('New template position #' + str(positionNumber))
    time.sleep(2)

def searchFinger():
    try:
        print('Waiting for finger...')
        while( f.readImage() == False ):
            #pass
            time.sleep(.5)
            return
        f.convertImage(0x01)
        result = f.searchTemplate()
        positionNumber = result[0]
        accuracyScore = result[1]
        if positionNumber == -1 :
            print('No match found!')
            time.sleep(2)
            return
        else:
            print('Found finger at position #' + str(positionNumber))
            time.sleep(2)

    except Exception as e:
        print('Operation failed!')
        print('Exception message: ' + str(e))
        exit(1)

def deleteFinger():
    positionNumber = 0
    count=0
    while gpio.input(delet) == True:   # here delet key means ok
      positionNumber = input('Please enter the template position you want to delete: ')
      positionNumber = int(positionNumber)
      if f.deleteTemplate(positionNumber) == True :
        print('Template deleted!')
        time.sleep(1)
        print('Currently used finger templates: ' + str(f.getTemplateCount()) +'/'+ str(f.getStorageCapacity()))
        time.sleep(1)
        return


print ("Edkits Electronics Welcomes You ")
time.sleep(3)
flag=0
```

EDKITS

```
while 1:
   gpio.output(led, HIGH)

   if gpio.input(enrol) == 0:
      gpio.output(led, LOW)
      enrollFinger()
   elif gpio.input(delet) == 0:
      gpio.output(led, LOW)
      while gpio.input(delet) == 0:
         time.sleep(0.1)
      deleteFinger()
   else:
      searchFinger()
```

It's fairly long but it will help you set-up and test your sensor. Now Run

Fingprint2.py

For Enrollment / Delete of fingerprint follow the given step.

1. **Connect Raspberry Pi Pin 29 to GND (Pin 6) once and follow onscreen instruction.**

For Delete already enroll fingerprint.

2. **Connect Raspberry Pi Pin 31 to GND (Pin 6) once and enter the position of Enroll fingerprint to delete it.**

EDKITS

GPS Module Interfacing with Raspberry Pi

Hardware Guide:

For completing this lesson, you will require the following things along with your initial raspberry pi setup

1. GPS module
2. USB to TTL converter
3. Connecting wires

GPS Module:

Global Positioning System (GPS) makes use of signals sent by satellites in space and ground stations on Earth to accurately determine their position on Earth.

Radio Frequency signals sent from satellites and ground stations are received by the GPS. GPS makes use of these signals to determine its exact position.

The signals received from the satellites and ground stations contain time stamps of the time when the signals were transmitted.

Using information from 3 or more satellites, the exact position of the GPS can be triangulated.



GPS receiver module gives output in standard (National Marine Electronics Association) NMEA string format. It provides output serially on Tx pin with default 9600 Baud rate.

This NMEA string output from GPS receiver contains different parameters separated by commas like longitude, latitude, altitude, time etc. Each string starts with '$' and ends with carriage return/line feed sequence.

E.g.

$GPGGA,184237.000,1829.9639,N,07347.6174,E,1,05,2.1,607.1,M,-64.7,M,,0000*7D

$GPGSA,A,3,15,25,18,26,12,,,,,,,,5.3,2.1,4.8*36

$GPGSV,3,1,11,15,47,133,46,25,44,226,45,18,37,238,45,26,34,087,40*72

NOTE: IF YOU WANT TO RUN PYTHON CODE FIRST INSTALL THE REQUIRED LIBRARIES ON YOUR PI4.

pip3 install pyserial pynmea2

After installation please run the code.

Sudo python3 GPS.py

It will takes 15 to 20 minutes to show the result.

USING BUILT IN UART

Enabling Built in UART

For enabling built in UART refer to page No. 28 of fingerprint section of this manual.

Wiring up your circuit:

In this Raspberry Pi GPS Module interfacing project we connect GPS sensor to Raspberry Pi UART. So, first of all, we need to make the all the required connection as shown below.
Connections are simple, we have just connected GPS module to Raspberry Pi UART port

1. Connect the VCC Pin of GPS Module to 3.3V Pin Pin 1 of Raspberry Pi.
2. Connect the GND Pin of GPS Module to GND Pin 6 of Raspberry Pi.
3. Connect the Tx Pin of GPS Module to Rx Pin 10 of Raspberry Pi.
4. Connect the RXD Pin GPS Module TXD Pin 8 of Raspberry Pi.

Software Guide:

We can find whether our GPS module is working properly and the connections are correct by typing the following command: sudo cat /dev/ttyS0

USING USB TO TTL Converter Module

Wiring up your Circuit:

1. Connect the VCC Pin of GPS Module to 3.3V Pin of USB to TTL converter
2. Connect the GND Pin of GPS Module to GND Pin of USB to TTL converter
3. Connect the Tx Pin of GPS Module to Rx Pin of USB to TTL converter
4. Connect the Rx Pin of GPS Module to Tx Pin of USB to TTL converte r.
5. Lastly connect the USB to TTL converter to USB port of Raspberry Pi.

Software Guide:

Open Terminal Window and type the following command to know to which USB port the GPS module is attached: ls /dev/ttyUSB*
We can find whether our GPS module is working properly and the connections are correct by typing the following command: sudo cat /dev/ttyUSB*

Use 'gpsd':

You can always just read that raw data, but its much nicer if you can hav e some Linux software prettify it. We'll try out gpsd which is a GPS-handling Daemon (background-helper)

Installing a GPS Daemon (gpsd)

The first step is installing some software on your Raspberry Pi that understands the serial data that your GPS module is providing via /dev/ttyUSB0.

Thankfully other people have already done all the hard work for you of properly parsing the raw GPS data, and we can use (amongst other options) a nice little package named 'gpsd', which essentially acts as a layer between your applications and the actual GPS hardware, gracefully handling parsing errors, and providing a common, well-defined interfaces to any GPS module.

To install gpsd, make sure your Pi has an Internet connection and run the following commands from the console:

1. sudo apt-get update
2. sudo apt-get install gpsd gpsd-clients python-gps

And install the software as it prompts you to do.

EDKITS

Raspbian Jessie systemd service fix:

Note if you're using the Raspbian Jessie or later release you'll need to disable a systemd service that gpsd installs. This service has systemd listen on a local socket and run gpsd when clients connect to it, however it will also interfere with other gpsd instances that are manually run (like in this guide). You will need to disable the gpsd systemd service by running the following commands:

1. sudo systemctl stop gpsd.socket
2. sudo systemctl disable gpsd.socket

Should you ever want to enable the default gpsd systemd service you can run these commands to restore it (but remember the rest of the steps in this guide won't work!):

1. sudo systemctl enable gpsd.socket
2. sudo systemctl start gpsd.socket

Try out 'gpsd'

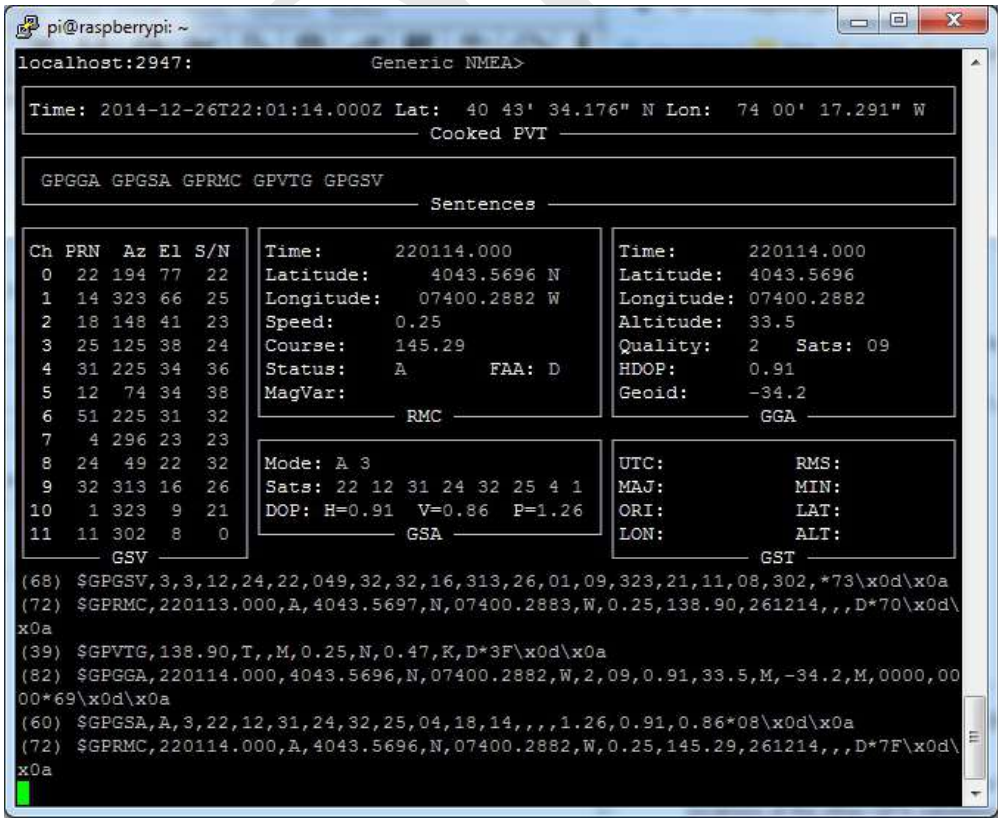
After installing gpsd and disabling the gpsd systemd service as mentioned above you're ready to start using gpsd yourself.

Start gpsd and direct it to use USB. Simply entering the following command(Here we are assuming that GPS module is connected to USB0):

1. sudo gpsd /dev/ttUSB0 -F /var/run/gpsd.sock

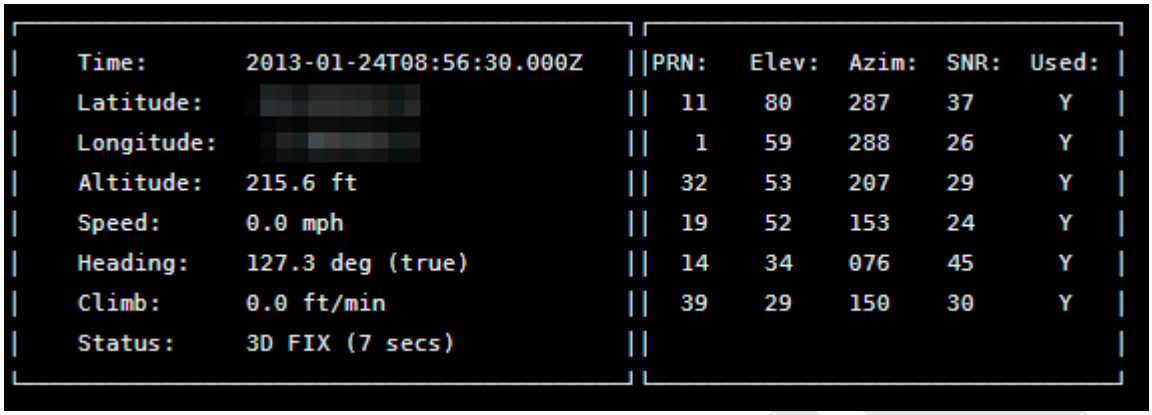
... which will point the gps daemon to our GPS device on the /dev/ttyAMA0 console

Try running **gpsmon** to get a live-streaming update of GPS data!



or cgps which gives a less detailed, but still quite nice output

1. cgps -s



You can abort gpsd by the following command

1. sudo killall gpsd

Using Python

Let's extract Latitude, Longitude and time information from NME! GPGG! string received from GPS module using Python. And print them on console (terminal). By using these latitude and longitude, locate the current position on Google Map.

Code:

```
'''
GPS Interfacing with Raspberry Pi using Pyhton
http://www.electronicwings.com
'''
import serial           #import serial pacakge
from time import sleep
import webbrowser       #import package for opening link in browser
import sys              #import system package

def GPS_Info():
    global NMEA_buff
    global lat_in_degrees
    global long_in_degrees
    nmea_time = []
    nmea_latitude = []
    nmea_longitude = []
    nmea_time = NMEA_buff[0]                #extract time from GPGGA string
    nmea_latitude = NMEA_buff[1]            #extract latitude from GPGGA string
    nmea_longitude = NMEA_buff[3]           #extract longitude from GPGGA string

    print("NMEA Time: ", nmea_time,'\n')
```

EDKITS

```
    print ("NMEA Latitude:", nmea_latitude,"NMEA Longitude:", nmea_longitude,'\n')

    lat = float(nmea_latitude)              #convert string into float for calculation
    longi = float(nmea_longitude)           #convertr string into float for calculation
    lat_in_degrees = convert_to_degrees(lat)    #get latitude in degree decimal format
    long_in_degrees = convert_to_degrees(longi) #get longitude in degree decimal format

#convert raw NMEA string into degree decimal form at
def convert_to_degrees(raw_value):
    decimal_value = raw_value/100.00
    degrees = int(decimal_value)
    mm_mmmm = (decimal_value - int(decimal_value))/0.6
    position = degrees + mm_mmmm
    position = "%.4f" %(position)
    return position



gpgga_info = "$GPGGA,"
ser = serial.Serial ("/dev/ttyUSB0")          #Open port with baud rate
GPGGA_buffer = 0
NMEA_buff = 0
lat_in_degrees = 0
long_in_degrees = 0

try:
    while True:
        received_data = (str)(ser.readline ())              #read NMEA string received
        GPGGA_data_available = received_data.find(gpgga_info)   #check for NMEA GPGGA string
        if (GPGGA_data_available>0):
            GPGGA_buffer = received_data.split("$GPGGA,",1)[1]  #store data coming after "$GPGGA,"
            NMEA_buff = (GPGGA_buffer.split(','))           #store comma separated data in buffer
            GPS_Info()                                  #get time, latitude, longitude

            print("lat in degrees:", lat_in_degrees," long in degree: ", long_in_degrees, '\n')
            map_link = 'http://maps.google.com/?q=' + lat_in_degrees + ',' + long_in_degrees
#create link to plot location on Google map
            print("<<<<<<<<press ctrl+c to plot location on google maps>>>>>>\n")
#press ctrl+c to plot on map and exit
            print("------------------------------------------------------------\n")

except KeyboardInterrupt:
    webbrowser.open(ma p_link)       #open current position information in google map
    sys.exit(0)

#end of file
```

EDKITS

You can download or copy the code from the following link:
http://www.electronicwings.com/raspberry-pi/gps-module-interfacing-with-raspberry-pi

The output of python code is as follows:



Note: Please ensure that GPS module is visible to open sky or else it will not be able to produce desired output.

IOT based Web Controlled Home Automation using Raspberry Pi

Hardware Guide:

For completing this lesson, you will require the following things along with your initial raspberry pi setup:

1. Relay Module
2. An Appliance
3. Connecting wires

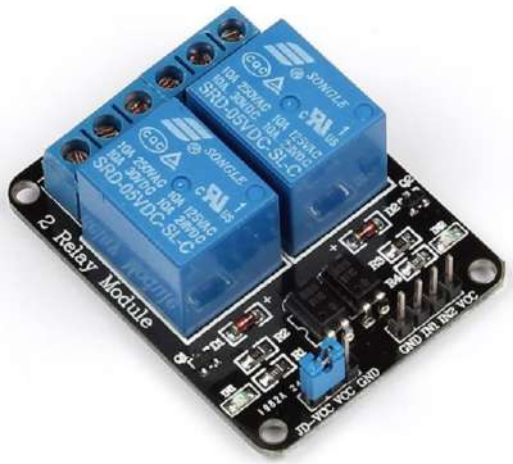
Relay Module

This is ready breakout board with two 5V activated relays, and the signal is opto -isolated, it is protected. Easy to use and suitable for beginner, and of course student.

If you need to control AC or high current, high voltage load, this will be the perfect board.

Features:

1. 5V 2 channel relay interface board
2. Maximum Current Rating: 10A
3. Maximum Voltage Rating: AC 250V / DC 30V
4. Can control various appliances with large current and high voltage
5. Can be controlled directly by microcontroller such as Arduino, 8051, AVR, PIC, DSP and ARM
6. Build in 2 units of Opto-Isolator IC 817C.



EDKITS

Software Guide:

Now before wiring up your circuit, you need to install WebIOPi framework which will help us handle communication from the webpage to the raspberry pi

Installation of WebIOPi:

To update the raspberry Pi below commands and then reboot the RPi :

1. sudo apt-get update
2. sudo reboot

With this done, the next thing is for us to install the webIOPi framework.
Make sure you are in home directory using;

1. cd ~

Use wget to get the file from their sourceforge page;
wget http://sourceforge.net/projects/webiopi/files/WebIOPi-0.7.1.tar.gz

When download is done, extract the file and go into the directory;

1. tar xvzf WebIOPi-0.7.1.tar.gz
2. cd WebIOPi-0.7.1/

At this point before running the setup, we need to install a patch as this version of the WebIOPi does not work with the raspberry pi 3 which I am using and I couldn't find a version of the WebIOPi that works expressly with the Pi 3.

Below commands are used to install patch while still in the WebIOPi directory, run :

1. wget https://raw.githubusercontent.com/doublebind/raspi/master/webiopi-pi2bplus.patch
2. patch -p1 -i webiopi-pi2bplus.patch
3. sudo chmod 775 setup.sh

Then we can run the setup installation for the WebIOPi using :

1. sudo ./setup.sh

Keep saying yes if asked to install any dependencies during setup installation. When done, reboot your pi:

1. sudo reboot

Test Webiopi Installation:

Before jumping in to schematics and codes, With the Raspberry Pi back on, we will need to test our WebIOPi installation to be sure everything works fine as desired.

Run the command at terminal window;

sudo /etc/init.d/webiopi start

EDKITS

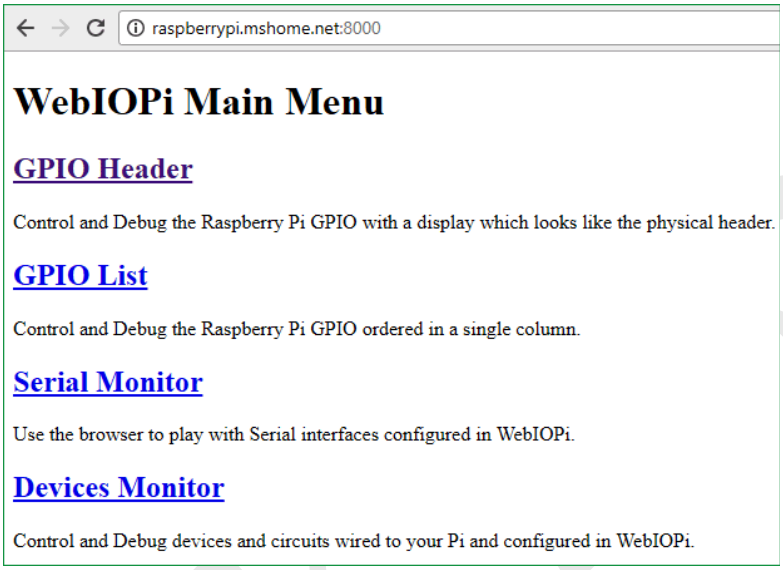
After issuing the command above on the pi, point the web browser of your computer connected to the raspberry pi to http;//thepi'sIPaddress:8000. The system will prompt you for username and password. Default username is 'webiopi' and password is 'raspberry'

After the login, look around, and then click on the GPIO header link.

NOTE:

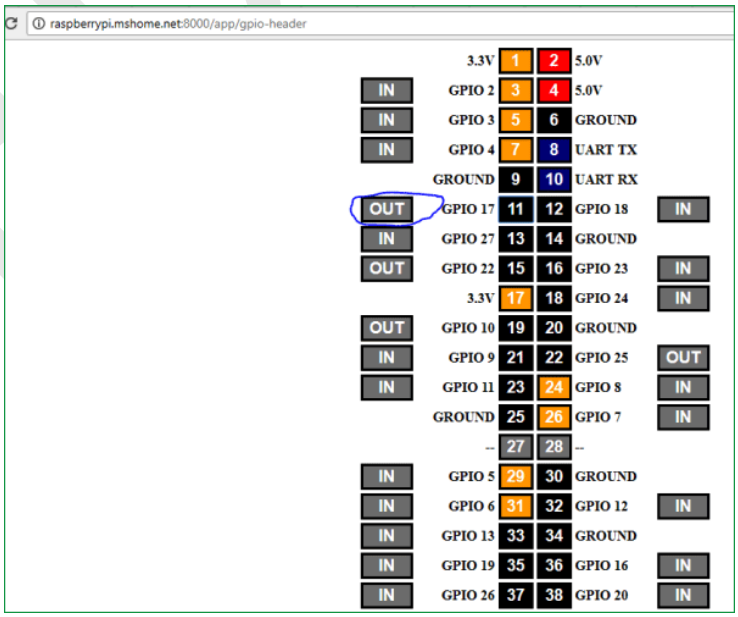
You can also start/stop the background service, the configuration will be loaded from /etc/webiopi/config.

1. sudo /etc/init.d/webiopi start
2. sudo /etc/init.d/webiopi stop

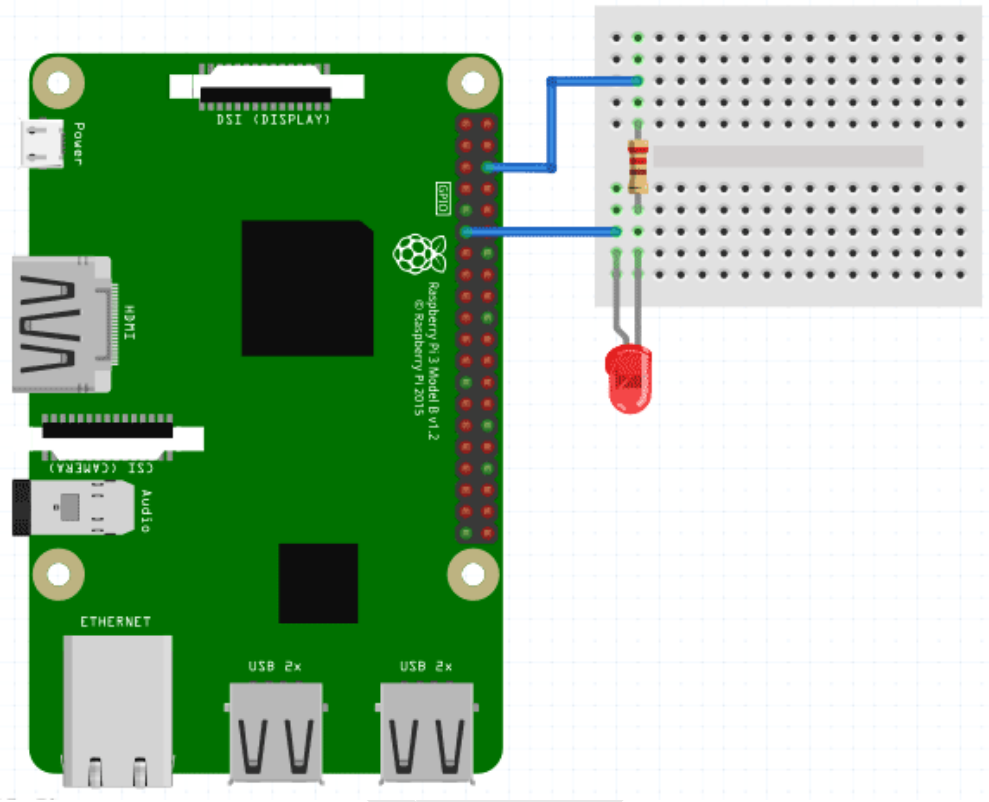


Wiring up your circuit:

For this test, we will be connecting an LED to GPIO 17, so go on and set GPIO 17 as an output.



With this done, connect the led to your raspberry pi as shown in the schematics below.



After the connection, go back to the webpage and click the OUT mode next to Pin 11. Now you can click on pin 11 turn on or off the LED. This way we can control the Raspberry Pi GPIO using WebIOPi.

After the test, if everything worked as described, then we can go back to the terminal and stop the program with CTRL + C.

Once everything is working properly you can connect the relay module to raspberry pi as follows:

1. Connect the positive of external 5v supply to the VCC pin of relay module.
2. Connect the negative of external 5v supply to the GND pin of relay module.
3. Also connect the GND pin of relay module to any GND pin of raspberry pi
4. Lastly connect IN1 and IN2 pins of relay module to any GPIO pins of raspberry which you need to use.
5. Now carefully connect IoT Box with Lamp to Relay board and verify that you can turn on or off the lamp with webiopi.

Note:

You can get more information on WebIOPi framework on the following link:
http://webiopi.trouch.com/

Camera Connection and capturing Images

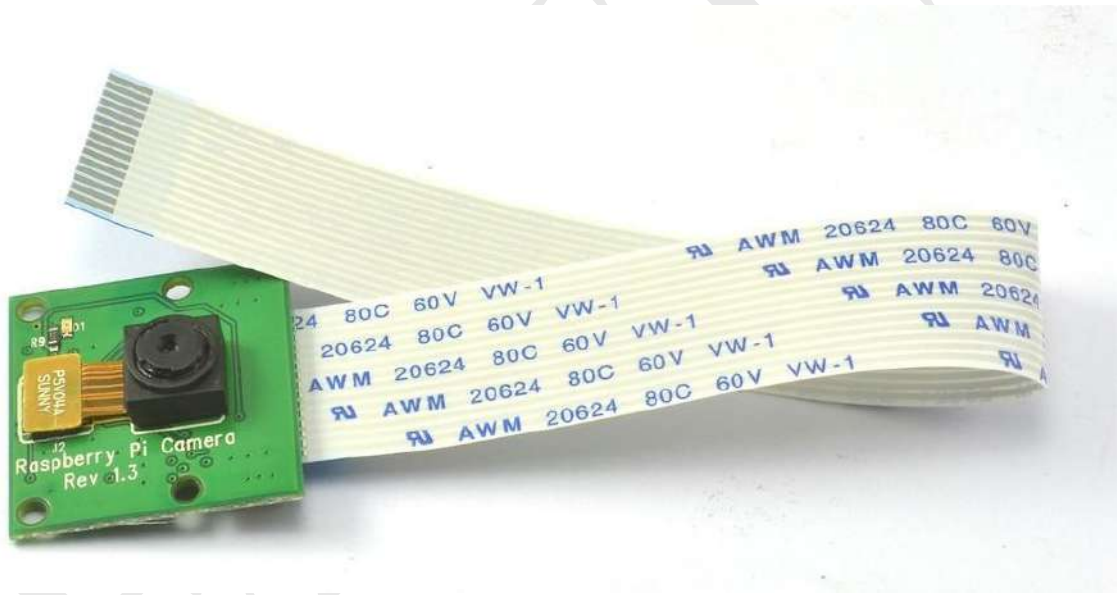
The Camera Module is a great accessory for the Raspberry Pi, allowing users to take still pictures and record video in full HD.

Hardware Guide:

For completing this lesson, you will require the Camera Module along with your initial raspberry pi setup.

Camera Module:

The Raspberry Pi Camera Board plugs directly into the CSI connector on the Raspberry Pi. The camera is supported in the latest version of Raspbian, the Raspberry Pi's preferred operating system.



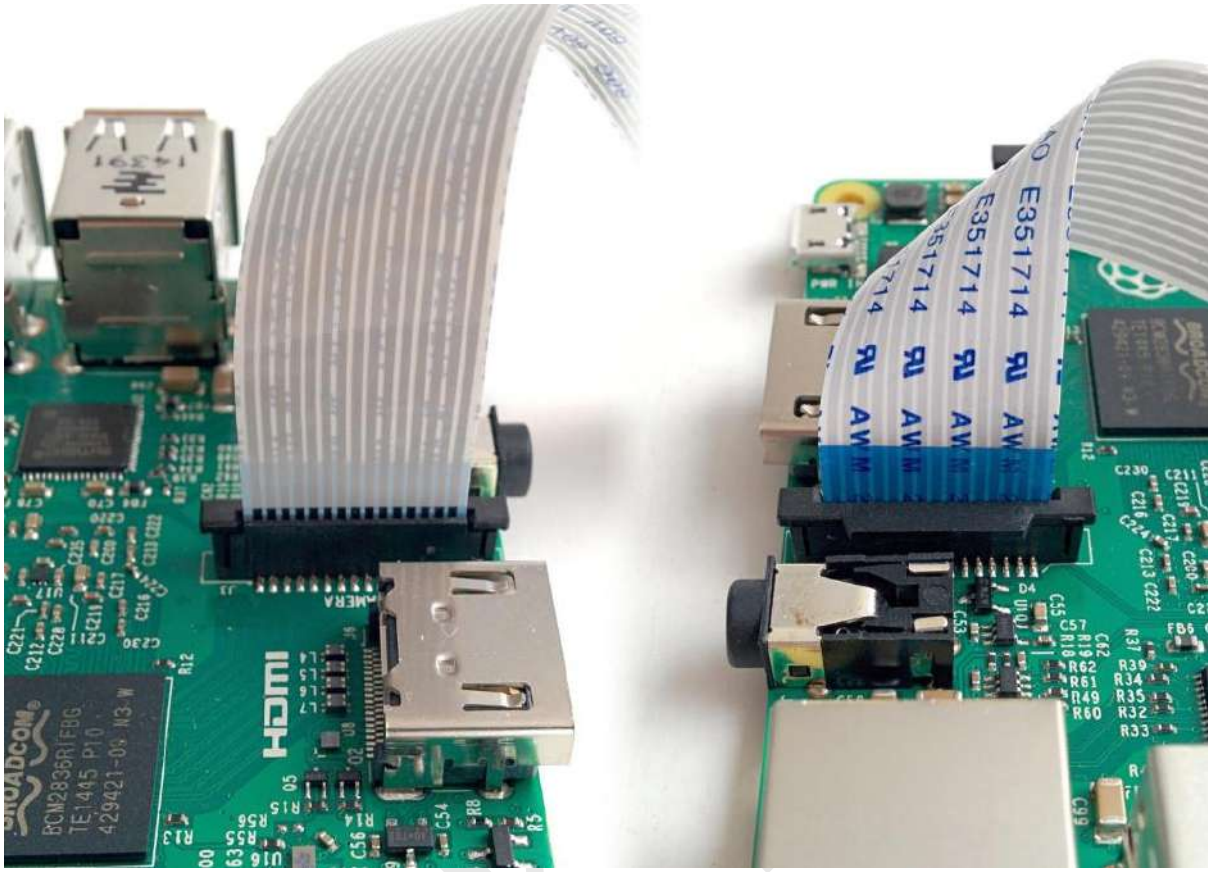
The Raspberry Pi Camera Board Features:

1. Fully Compatible with Both the Model A and Model B Raspberry Pi
2. 5MP Omnivision 5647 Camera Module
3. Still Picture Resolution: 2592 x 1944
4. Video: Supports 1080p @ 30fps, 720p @ 60fps and 640x480p 60/90 Recording
5. 15-pin MIPI Camera Serial Interface – Plugs Directly into the Raspberry Pi Board
6. Size: 20 x 25 x 9mm
7. Weight 3g
8. Fully Compatible with many Raspberry Pi cases

Connect the Camera Module:

First of all, with the Pi switched off, you'll need to connect the Camera Module to the Raspberry Pi's camera port, then start up the Pi and ensure the software is enabled.

1. Locate the camera port and connect the camera:



Software Guide:

Now your camera is connected and the software is enabled, you can get started by capturing an image.

You can capture an image by just typing a single line command. Open terminal window and type the command as follows:

$ sudo raspistill- libcamera-jpeg -o test.jpg

This command will capture an image and store it at Home directory

Use the following options to take a picture, called test1080.jpg with a preview delay of five seconds, and an image size of 1920 x 1080. Note that the time -t is specified in milliseconds.

libcamera-jpeg -o test1080.jpg -t 5000 --width 1920 --height 1080

Advanced Options

The more advanced way to capture images is via libcamera-still. This command shares a similarity to raspistill, in that many of the same arguments work across the pair.

1. Open a terminal and enter the command to start the camera, take a picture and save it as still-test.jpg.

libcamera-still -o still-test.jpg

2. To capture a png image, use the -e switch to specify the encoding and ensure the filename ends in .png. We can change the encoding to png, bmp, rgb and yuv420.

libcamera-still -e png -o still-test.png

3. Use the --datetime switch to capture an image. This will save the image as a JPG using the MMDDhhmmss date format as a filename.

libcamera-still --datetime

Capturing Video :

Using libcamera-vid, we can capture high definition video in h264, mjpeg and yuv420 formats.

1. Open a terminal and enter the command to start the camera, record a ten second video and save it as test.h264. You will notice that the switches, -t and -o work in the same manner as with libcamera-still. We can use the --codec switch to use mjpeg or yuv420 output formats.

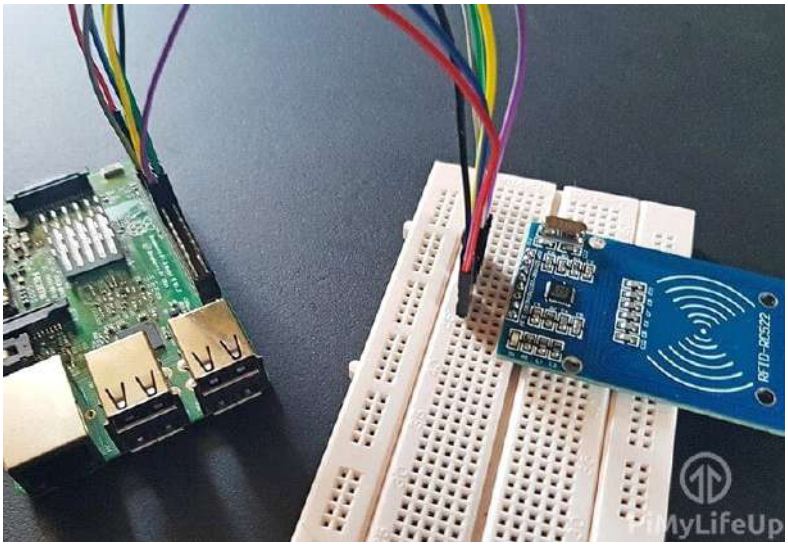
libcamera-vid -t 10000 -o test.h264

2. Go to the folder containing the file (/home/pi) and right click on test.h264 and select VLC Media Player to play the file. The Raspberry Pi camera doesn't have a microphone so there will be no sound.

EDKITS

Interfacing Raspberry Pi With RFID RC522 Chip

In this Raspberry Pi RFID RC522 tutorial, I will be walking you through the steps on how to set up and wire the RFID RC522 chip with your Raspberry Pi.



The RFID RC522 is a very low-cost RFID (Radio-frequency identification) reader and writer that is based on the MFRC522 microcontroller.

This microcontroller provides its data through the SPI protocol and works by creating a 13.56MHz electromagnetic field that it uses to communicate with the RFID tags.

Make sure that the tags you purchase for your RFID RC522 operate on the 13.56MHz frequency otherwise we will fail to read them.

We will be showing you how to wire up the RC522 as well as showing you how to write Python scripts to interact with the chip so you can both read and write your RFID Tags.

EQUIPMENT LIST:

Below are all the bits and pieces that I used for this Raspberry Pi RFID RC522 tutorial.

Recommended

- Raspberry Pi
- Micro SD Card
- Power Supply
- RC522 RFID Reader
- Breadboard
- Breadboard Wire

Optional

- Raspberry Pi Case
- Ethernet Cable or Wi-Fi

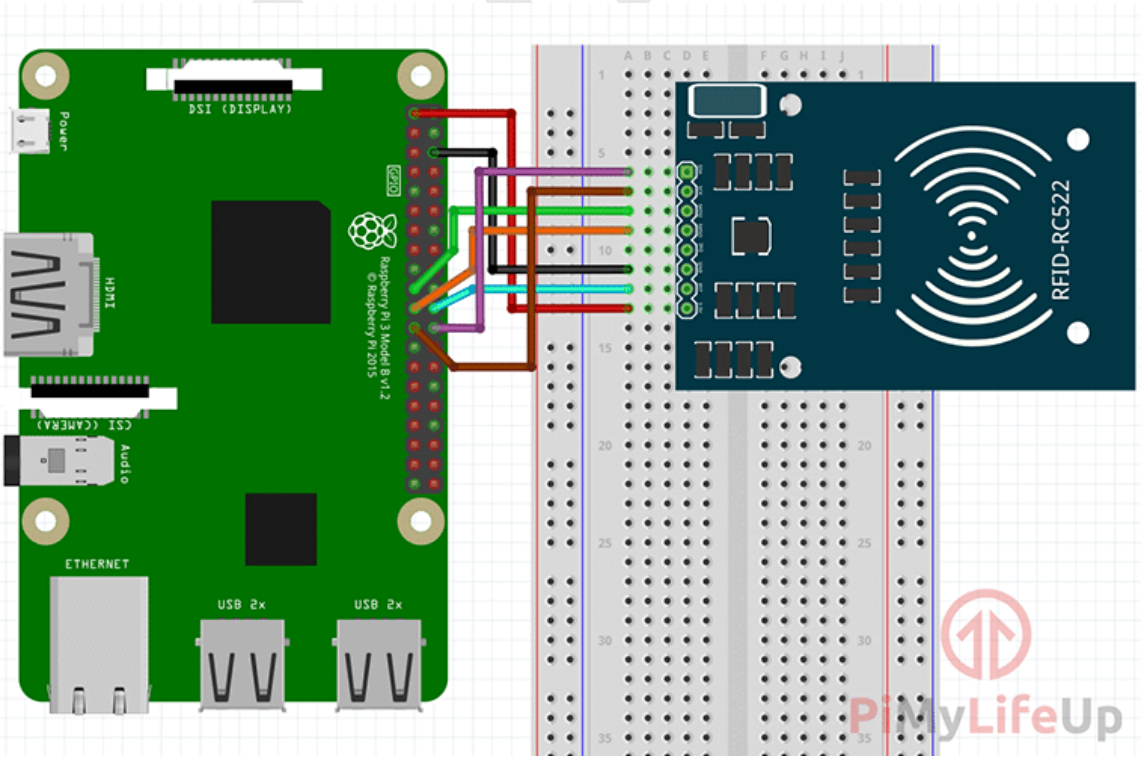
Wiring the RFID RC522

On your RFID RC522 you will notice that there are 8 possible connections on it, these being **SDA** (Serial Data Signal), **SCK** (Serial Clock), **MOSI** (Master Out Slave In), **MISO** (Master In Slave Out), **IRQ** (Interrupt Request), **GND** (Ground Power), **RST** (Reset-Circuit) and **3.3v** (3.3v Power In). We will need to wire all of these but the **IRQ** to our Raspberry Pi's GPIO pins.

You can either wire these directly to the GPIO Pins or like we did in this tutorial, plug the RFID RC522 into our Breadboard then wire from there to our Raspberry Pi's GPIO Pins.

Wiring your RFID RC522 to your Raspberry Pi is fairly simple, with it requiring you to connect just 7 of the GPIO Pins directly to the RFID reader. Follow the table below, and check out our GPIO guide to see the positions of the GPIO pins that you need to connect your RC522 to.

- **SDA** connects to **Pin 24**.
- **SCK** connects to **Pin 23**.
- **MOSI** connects to **Pin 19**.
- **MISO** connects to **Pin 21**.
- **GND** connects to **Pin 6**.
- **RST** connects to **Pin 22**.
- **3.3v** connects to **Pin 1**.



Setting up Raspbian for the RFID RC522

Before we begin the process of utilizing the RFID RC522 on our Raspberry Pi, we will first have to make changes to its configuration. By default, the Raspberry Pi has the SPI (Serial Peripheral Interface) disabled, which is a bit of a problem as that is what our RFID reader circuit runs through.

Don't worry though as it is fairly simple to re-enable this interface, just follow our steps below to configure your Raspberry Pi and Raspbian to utilize the SPI interface.

1. Let's begin by first opening the **raspi-config** tool, and we can do this by opening the terminal and running the following command.

```
sudo raspi-config
```

2. This tool will load up a screen showing a variety of different options. If you want a more in-depth look into these options, you can check out our raspi-config guide.

On here use the **arrow keys** to select "**5 Interfacing Options**". Once you have this option selected, press **Enter**.

3. Now on this next screen, you want to use your **arrow keys** to select "**P4 SPI**", again press Enter to select the option once it is highlighted.

4. You will now be asked if you want to enable the SPI Interface, select **Yes** with your **arrow keys** and press **Enter** to proceed. You will need to wait a little bit while the **raspi-**config tool does its thing in enabling SPI.

5. Once the SPI interface has been successfully enabled by the **raspi-config** tool you should see the following text appear on the screen, "**The SPI interface is enabled**".

Before the SPI Interface is fully enabled we will first have to restart the Raspberry Pi. To do this first get back to the terminal by pressing **Enter** and then **ESC**.

Type the following Linux command into the terminal on your Raspberry Pi to restart your Raspberry Pi.

```
sudo reboot
```

6. Once your Raspberry Pi has finished rebooting, we can now check to make sure that it has in fact been enabled. The easiest way to do this is to run the following command to see if **spi_bcm2835** is listed.

```
   lsmod | grep spi
```

If you see **spi_bcm2835**, then you can proceed on with this tutorial and skip on to the next section. If for some reason it had not appeared when you entered the previous command, try following the next three steps.

7. If for some reason the SPI module has not activated, we can edit the boot configuration file manually by running the following command on our Raspberry Pi.

```
sudo nano /boot/config.txt
```

8. Within the configuration file, use `CTRL + W` to find "**dtparam=spi=on**".

If you have found it, check to see if there is a **#** in front of it. If there is, remove it as this is commenting out the activation line. If you can't find the line at all, add "**dtparam=spi=on**" to the bottom of the file.

Once you have made the changes, you can press `CTRL + X` then pressing `Y` and then `Enter` to save the changes.

You can now proceed from **Step 5** again, rebooting your Raspberry Pi then checking to see if the module has been enabled.

Getting Python ready for the RFID RC522

Now that we have wired up our RFID RC522 circuit to the Raspberry Pi we can now power it on and begin the process of programming simple scripts in Python to interact with the chip.

The scripts that we will be showing you how to write will show you how to read data from the RFID chips and how to write to them. These will give you the basic idea of how data is dealt with and will be the basis of further RFID RC522 tutorials.

1. Before we start programming, we first need to update our Raspberry Pi to ensure it's running the latest version of all the software. Run the following two commands on your Raspberry Pi to update it.

```
sudo apt update
```

2. Now the final thing we need before we can proceed is to install **python3-dev**, **python-pip** and **git** packages. Simply run the following command on your Raspberry Pi to install all of the required packages for this guide on setting up your RFID reader.

```
sudo apt install python3-dev python3-pip
```

3. To begin, we must first install the Python Library spidev to our Raspberry Pi using the python "**pip**" tool that we downloaded in the previous step.

The **spidev** library helps handle interactions with the SPI and is a key component to this tutorial as we need it for the Raspberry Pi to interact with the RFID RC522.

Run the following command on your Raspberry Pi to install **spidev** to your Raspberry Pi through pip.

Please note that we use sudo here to ensure that the package is installed so that all users can utilize it and not just the current user.

```
sudo pip3 install spidev
```

4. Now that we have installed the **spidev** library to our Raspberry Pi we can now now proceed to installing the MFRC522 library using pip as well.

You can check out the RFID MFRC522 Python code from the PiMyLifeUp Github if you are interested in seeing how the library works or improving its behaviour .

There are two files that are included within our MFRC522 library that we make use of:

MFRC522.py which is an implementation of the RFID RC522 interface, this library handles all the heavy lifting for talking with the RFID over the Pi's SPI Interface.

SimpleMFRC522.py that takes the **MFRC522.py** file and greatly simplifies it by making you only have to deal with a couple of functions instead of several.

To install the MFRC522 library to your Raspberry Pi using pip go ahead and run the following command.

```
sudo pip3 install mfrc522
```

5. With the library now saved to our Raspberry Pi, we can begin programming for our RFID RC522. To start off with we will be showing you how to write data to your RFID cards by using the RC522. Simply go onto our next section to begin programming our first Python script.

Writing with the RFID RC522

For our first Python script, we will be showing you how to write data from the RC522 to your RFID tags. Thanks to the SimpleMFRC522 script this will be relatively simple, but we will still go into how each part of the code works.

1. Now lets start off by making a folder where we will be storing our couple of scripts.

We will be calling this folder "pi-rfid", create it by running the following command.

```
mkdir ~/pi-rfid
```

2. Begin by changing directory into our newly cloned folder, and begin writing our **Write.py** Python script.

```
cd ~/pi-rfid
sudo nano Write.py
```

3. Code

```
#!/usr/bin/env python

import RPi.GPIO as GPIO
from mfrc522 import SimpleMFRC522

reader = SimpleMFRC522()

try:
        text = input('New data:')
        print("Now place your tag to write")
        reader.write(text)
        print("Written")
finally:
        GPIO.cleanup()
```

Once you are happy that the code looks correct, you can save the file by pressing CTRL + X then pressing Y and then finally hitting ENTER.

5. Now that we have written our script, we will want to test it out. Before testing out the script make sure that you have an RFID tag handy. Once ready, type the following command into your Raspberry Pi's terminal.

```
sudo python3 Write.py
```

6. You will be asked to write in the new data, in our case we are going to just type in **edkitselectronics** as its short and simple. Press **Enter** when you are happy with what you have written.

7. With that done, simply place your RFID Tag on top of your RFID RC522 circuit. As soon as it detects it, it will immediately write the new data to the tag. You should see "**Written**" appear in your command line if it was successful.

You can look at our example output below to see what a successful run looks like.

```
pi@raspberrypi:~/pi-rfid $ sudo python3 Write.py
New data:edkitselectronics
Now place your tag to write
Written
```

8. You have now successfully written your **Write.py** script, and we can now proceed to show you how to read data from the RFID RC522 in the next segment of this tutorial.

Reading with the RFID RC522

Now that we have written our script to write to RFID tags using our RC522 we can now write a script that will read this data back off the tag.

1. Let's start off by changing the directory to make sure we are in the right place, and then we can run nano to begin writing our **Read.py** script.

```
cd ~/pi-rfid
sudo nano Read.py
```

2. Within this file, write the following lines of code. This script will basically sit and wait till you put your RFID tag on the RFID RC522 reader, it will then output the data it reads off the tag.

3. Code

```
#!/usr/bin/env python

import RPi.GPIO as GPIO
from mfrc522 import SimpleMFRC522

reader = SimpleMFRC522()

try:
        id, text = reader.read()
        print(id)
        print(text)
finally:
        GPIO.cleanup()
```

Once you are sure you have entered the code correctly, you can save the file by pressing `Ctrl + X` then pressing `Y` and then finally hitting `ENTER`.

4. Now that we have finally finished our Read.py script we need to test it out. Before we test out the script, grab one of the RFID tags that you want to read. Once that you are ready, type the following command into your Raspberry Pi's terminal.

```
sudo python3 Read.py
```

5. With the script now running, all you need to do is place your RFID Tag on top of your RFID RC522 circuit. As soon as the Python script detects the RFID tag being placed on top, it will immediately read the data and print it back out to you.

An example of what a successful output would look like is displayed below.

```
pi@raspberrypi:~/pi-rfid $ sudo python3 Read.py
827843705425
Pimylifeup
```

7. If you successfully receive data back from your Read.py script with the text that you pushed to the card using your Write.py script then you have successfully set up your Raspberry Pi to connect with your RFID RC522 Circuit.

You can find the entire details on the link:

https://pimylifeup.com/raspberry-pi-rfid-rc522/

EDKITS

Installing Windows 10 IOT Core on Raspberry Pi

Windows 10 IoT is a member of the Windows 10 family that brings enterprise-class power, security and manageability to the Internet of Things. It leverages Windows' embedded experience, ecosystem and cloud connectivity, allowing organizations to create their Internet of Things with secure devices that can be quickly provisioned, easily managed, and seamlessly connected to an overall cloud strategy.

Hardware Guide:

For getting started with windows 10 IOT, you will require the following hardware

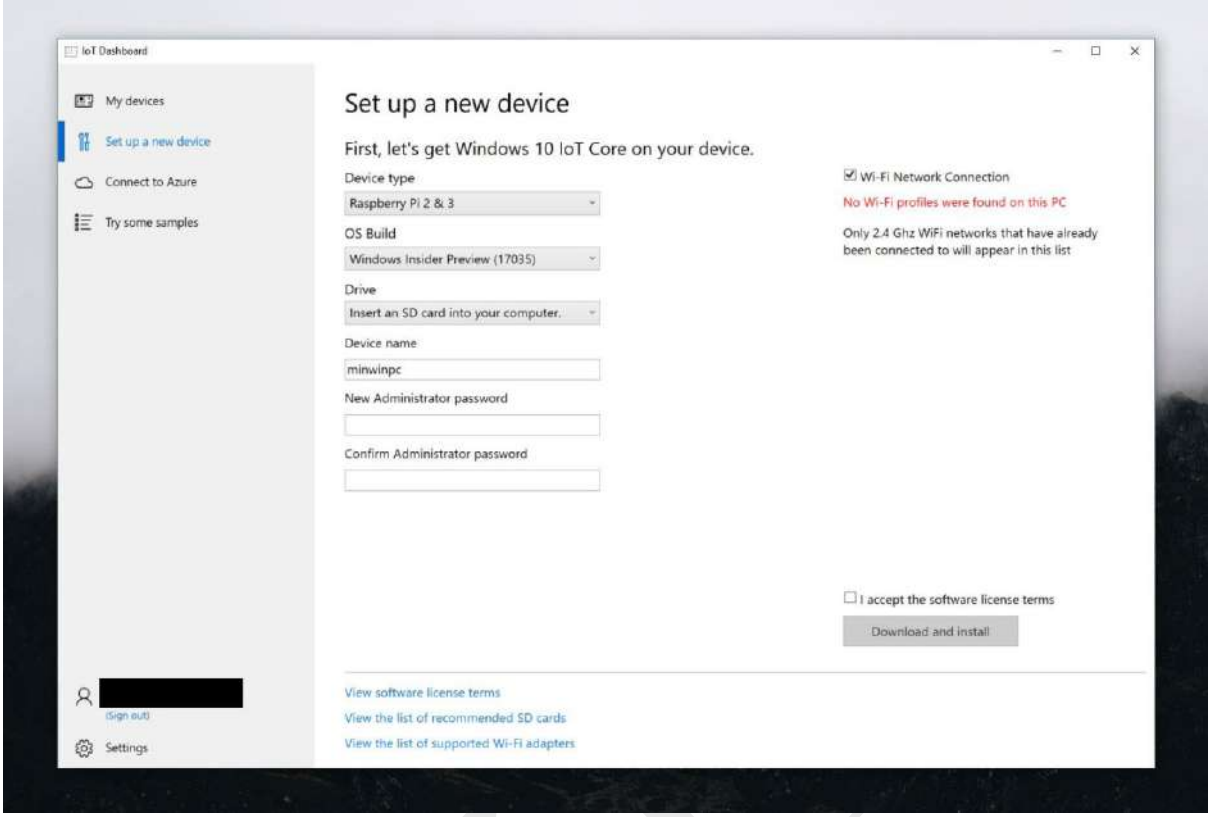
1. Raspberry Pi 3
2. 16 GB Micro SD Card – class 10
3. Display
4. Keyboard
5. Mouse
6. Windows 10 PC
7. Card Reader

Installation Guide:

Use this tutorial to get yourself comfortable with Windows 10 IoT quickly. You'll learn how to flash a Windows 10 IoT Core image onto a device and how to deploy an app from your device.

1. Download the Windows 10 IoT Core Dashboard from https://developer.microsoft.com/en-us/windows/iot/Downloads
2. Once downloaded, open the Dashboard and click on set up a new device and insert a SD card into your computer.
3. Fill out all of the fields as indicated.
4. Accept the software license terms and click Download and install. You'll see that Windows 10 IoT Core is now flashing onto your device.

EDKITS



Connecting to a network

Wired connection

If your device comes with an Ethernet port or USB Ethernet adapter support to enable a wired connection, attach an Ethernet cable to connect it to your network.

Wireless connection

If your device supports Wi-Fi connectivity and you've connected a display to it, you'll need to:

1. Go into your default application and click the settings button next to the clock.
2. On the settings page, select Network and Wi-Fi.
3. Your device will begin scanning for wireless networks.
4. Once your network appears in this list, select it and click Connect.

If you haven't connected and display and would like to connect via Wi-Fi, you'll need to:

1. Go to the IoT Dashboard and click on My Devices.
2. Find your unconfigured board from the list. Its name will begin with "AJ_"... (e.g. AJ_58EA6C68). If you don't see your board appear after a few minutes, try rebooting your board.
3. Click on Configure Device and enter your network credentials. This will connect your board to the network.

D.C. Motor Control with Raspberry Pi & L293D Driver

AIM:

Interfacing D.C. Motor with Raspberry pi & L293D Driver.

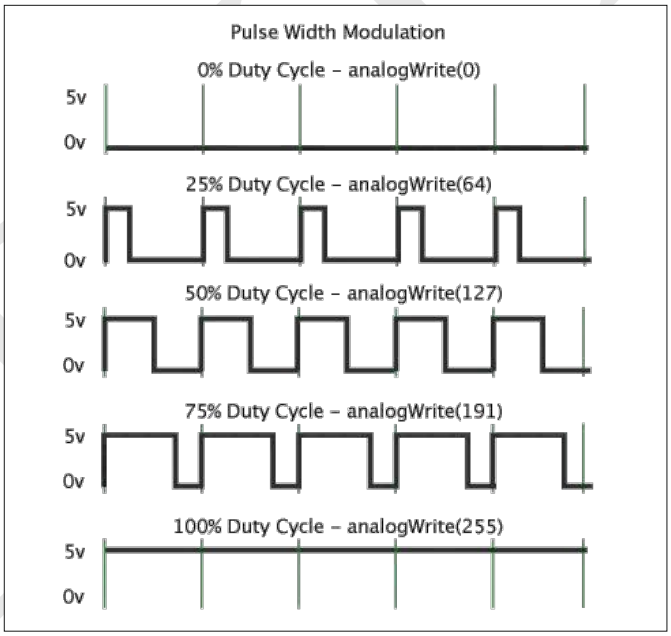
EQUIPMENT REQUIRED:

- Raspberry Pi Module
- L293D Driver
- Jumper wires
- DC Motor
- Integrated Kit / +5V Power Supply (External)

THEORY:

In this project, we're using python scripts run on a Raspberry Pi to set GPIO outputs to an L293D motor controller IC and run a DC motor in either direction at any speed.

The way we'll control the speed of the motor is by using a python module called PWM. That stands for Pulse Width Modulation. What PWM means is just controlling the amount of time a voltage is on by flipping between high and low for a set amount of time. The amount of time the voltage is high is called the 'duty' or 'duty cycle', and whatever percentage that is will be the percentage of power the motor runs on.



To see how PWM looks as an output, refer to the diagram above.

The L293D motor IC uses two pins referred to as inputs to sense the desired direction of the output, and another pin called Enable to sense On/Off. So, in our code, with the Enable pin On, if we want the motor to spin forward, we'll set input 1 to 'True' or 'HIGH', and input 2 to 'False' or 'LOW'. And if we want it to spin backwards, we'll set input 1 to 'False" or 'LOW' and input 2 to 'True' or "HIGH'. If both inputs are True or both are False, the motor will not run.

That's how we'll control the direction, but what about speed? We talked about PWM right? So we'll just Pulse Width Modulate both inputs, right? We could, but it would be too complicated. Since the IC has an Enable pin that controls its On/Off state, we can leave both inputs set to run and just modulate the Enable pin, and the IC will only put out power according to the duty we set in the Enable pin. That way we keep the code simpler, and less things can go wrong.

So, on the Raspberry Pi, we'll be using 3 GPIO output pins on the GPIO board, one 3.3V power supply, and one grounding pin. 2 of the output pins will be for the inputs on the IC, and one will be for the Enable.

Now that we know what will be going on, it's time to start wiring it up.

WIRING DETAILS:

1. Connect External supply 5V (positive) pin to VCC Pin of L293D Driver Module.
2. Connect GND Pin of External supply to GND Pin of L293D Driver Module.
3. Also Connect GND Pin of External Supply to Raspberry Pin GND (physical Pin 6).
4. Connect EB Pin of L293D Driver Module to Raspberry Pi GPIO 04 (physical Pin 7).
5. Connect B2 Pin of L293D Driver Module to Raspberry Pi GPIO 02 (physical Pin 3)
6. Connect B1 Pin of L293D Driver Module to Raspberry Pi GPIO 03 (physical Pin 5)

Everything is now wired exactly how it needs to be, no more hardware work. On to software. we're going to write a code that runs forward at 50% power for 2 seconds,the PWM stop for 1 seconds then runs backward at 75% power for 2 seconds.

PYTHON CODE :

```
import RPi.GPIO as GPIO #this imports the GPIO module
from time import sleep #we need a command called 'sleep'
GPIO.setmode(GPIO.BOARD)# This sets the names to board mode,
# now we need to set the pins as outputs, so write
GPIO.setup(3, GPIO.OUT)
GPIO.setup(5, GPIO.OUT)
GPIO.setup(7, GPIO.OUT)
#now to setup the pwm commands type
pwm=GPIO.PWM(7, 100)
#start the Pulse Width Modulation with 0 duty so it doesn't run yet
pwm.start(0)
#to set the direction to forward write
GPIO.output(3, True)
GPIO.output(5, False)
#Now, we're going to set the PWM duty to 50%. Write
pwm.ChangeDutyCycle(50)
#then turn on the Enable pin
GPIO.output(7, True)
sleep(2) #then put the code to sleep for 2 seconds so the motor runs
#now turn off the Enable pin
GPIO.output(7, False)
pwm.stop()
sleep(1)
#then reverse the inputs to set it to reverse
GPIO.output(3, False)
GPIO.output(5, True)
#Then change the PWM duty to 75%
pwm.ChangeDutyCycle(75)
#then turn the enable back on
GPIO.output(7, True)
sleep(2)
#then turn the enable pin back off
GPIO.output(7, False)
#stop the Pulse
pwm.stop()
#cleanup all of the GPIO channels.
GPIO.cleanup()
```

Oscilloscope Using Raspberry Pi

The oscilloscope is an electronic test instrument that allows the visualization and observation of varying signal voltages, usually as a two dimensional plot with one or more signals plotted against time. Today's project will seek to replicate the signal visualization capabilities of the oscilloscope using the Raspberry Pi and an analog to digital converter module.

Replicating the signal visualization of the oscilloscope using the Raspberry Pi will require the following steps;

1. Perform Digital to analog conversion of the Input signal
2. Prepare the resulting data for representation
3. Plot the data on a live time graph

Hardware requirements:

To build this project, the following components/part are required;

1. 1.Raspberry pi 4 (or any other model)
2. 8 or 16GB SD Card
3. LAN/Ethernet Cable
4. Power Supply or USB cable
5. ADS1115 ADC
6. 10k or 1k resistor
7. Jumper wires
8. Breadboard
9. Monitor or any other way of seeing the pi's Desktop(VNC inclusive)

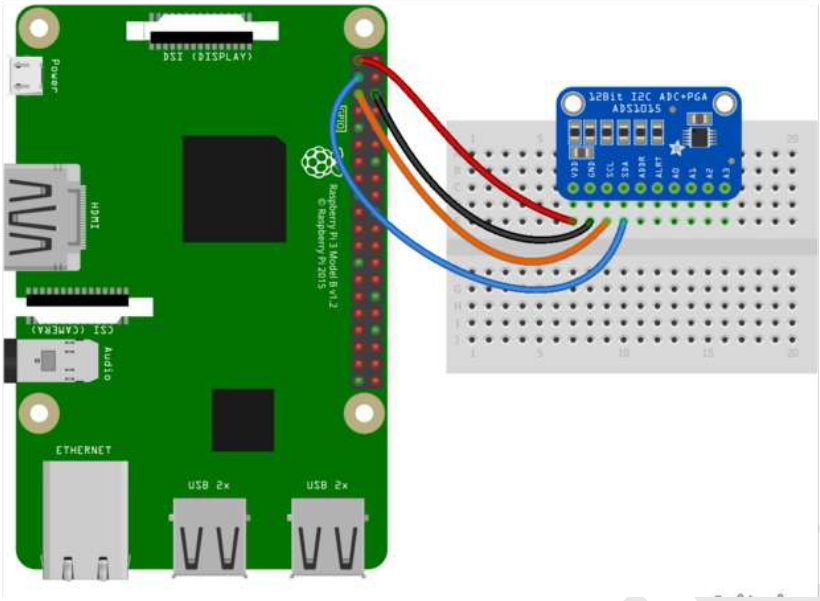
Wiring up your Circuit:

To convert the analog input signals to digital signals which can be visualized with the Raspberry Pi, we will be using the ADS1115 ADC chip. This chip becomes important because the Raspberry Pi, unlike Arduino and most micro-controllers, does not have an on-board analog to digital converter(ADC).

The ADC is an I2C based device and should be connected to the Raspberry Pi as shown in the schematics below.

For clarity, the pin connection between the two components is also described below.

EDKITS



ADS1115 and Raspberry Pi Wiring Details:

1. **Connect VDD Pin of ADS1115 Module to 3.3V (Pin 1) of Raspberry pi.**
2. **Connect GND Pin of ADS1115 Module to Ground (Pin 6) of Raspberry pi.**
3. **Connect SCL Pin of ADS1115 Module to SCL (Pin 5) of Raspberry pi.**
4. **Connect SDA – Pin of ADS1115 Module to SDA (Pin 3) of Raspberry pi.**
5. **Connect External +5V to One side of Potentiometer (10K).**
6. **Connect another side of Potentiometer to ground of External Supply and also with Raspberry pin Ground (Pin 39).**
7. **Connect Middle Pin of Potentiometer to A0 of ADS1115 Module.**

With the connections all done, power up your pi and proceed to install the dependencies mentioned below.

Software Guide:

Install Dependencies for Raspberry Pi Oscilloscope:

Before we start writing the python script to pull data from the ADC and plot it on a live graph, we need to enable the I2C communication interface of the raspberry pi and install the software requirements that were mentioned earlier. This will be done in below steps so its easy to follow:

Step 1: Enable Raspberry Pi I2C interface

sudo raspi-config

When the configuration panels open, select interface options, select I2C and click enable.

Step 2: Update the Raspberry Pi

The first thing I do before starting any project is updating the Pi. Through this, I am sure every thing on the OS is up to date and I won't experience compatibility issue with any latest software I choose to install on the Pi. To do this, run below two commands:

sudo apt-get update

EDKITS

Step 3: Install the Adafruit ADS1115 library for ADC

Ensure you are in the Raspberry Pi home directory by running;

cd ~

You'll need to install the Adafruit_Blinka library that provides the CircuitPython support in Python.

sudo pip3 install Adafruit-Blinka

With installing Adafruit _blinka done, we are now ready to install the dependencies starting with the Adafruit python module for the ADS115 chip.

sudo pip3 install adafruit-circuitpython-ads1x15

Step 4: Test the library and 12C communication.

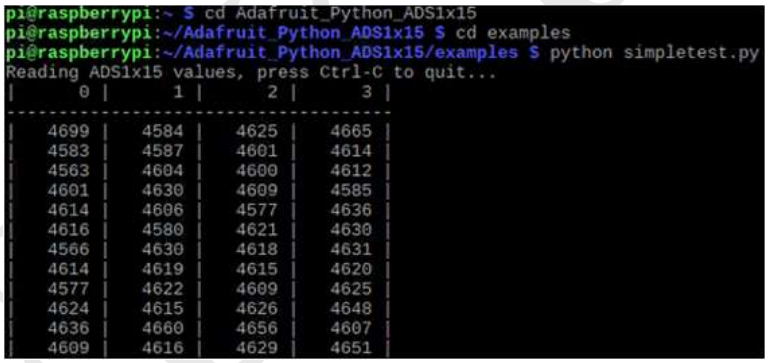
Before we proceed with the rest of the project, it is important to test the library and ensure the ADC can communicate with the raspberry pi over I2C. To do this we will use an example script that is with in home directory.

Next, run the simpletest.py example which displays the value of the four channels on the ADC in a tabular form.

Run the example file which is in located ADS_1115 folder using:

sudo python3 pi@raspberrypi:~/ADS_1115/simpletest.py

If the I2C module is enabled and connections good, you should see the data as shown in the image below.



NOTE : Vary the potentiometer to get value / voltage change in the above channels.

If an error occurs, check to ensure the ADC is well connected to the PI and I2C communication is enabled on the Pi.

EDKITS

Step 5: Install Matplotlib

To visualize the data we need to install the matplotlib module which is used to plot all kind of graphs in python. This can be done by running;

sudo apt-get install python3-matplotlib

Step 6: Install the Drawnow python module

Lastly, we need to install the drawnow python module. This module helps us provide live updates to the data plot.

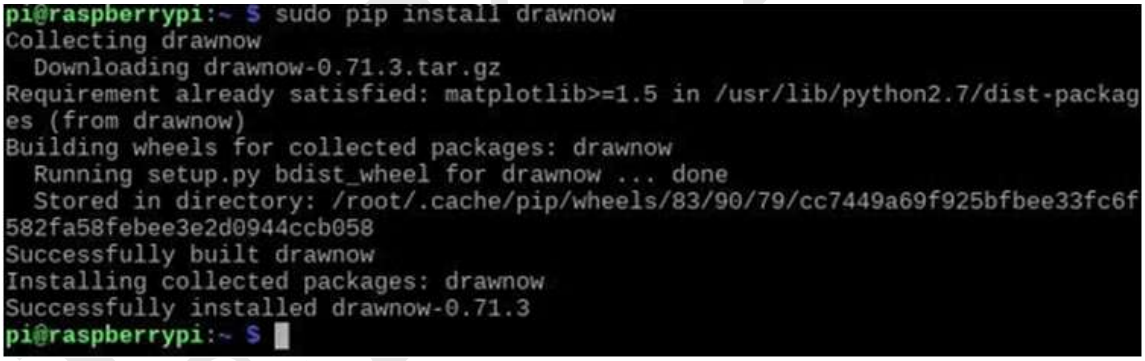
We will be installing drawnow via the python package installer; pip, so we need to ensure it is installed. This can be done by running;

sudo apt-get install python3-pip

We can then use pip to install the drawnow package by running:

sudo pip3 install drawnow

You should get an outcome like the image below after running it.



With all the dependencies installed, we are now ready to write the code.

Python Code for Raspberry Pi Oscilloscope:

The python code for this Pi Oscilloscope is fairly simple especially if you are familiar with the python matplotlibmodule. Before showing us the whole code, I will try to break it into part and explain what each part of the code is doing so you can have enough knowledge to extend the code to do more stuffs.

At this stage it is important to switch to a monitor or use the VNC viewer, anything through which you can see your Raspberry Pi's desktop, as the graph being plotted won't show on the terminal.

With the monitor as the interface open a new python file. You can call it any name you want, but I will call it scope.py.

sudo nano scope.py

Code

```
import time
import board
import matplotlib.pyplot as plt
from drawnow import drawnow
from adafruit_ads1x15.ads1115 import ADS1115
from adafruit_ads1x15.analog_in import AnalogIn
# ADS1115 SETUP
i2c = board.I2C()
ads = ADS1115(i2c)
ads.gain = 1
# Channel 0
chan = AnalogIn(ads, 0)
# GRAPH SETUP
val = []
plt.ion()
print("Reading ADS1115 Channel 0")
print("Press Ctrl+C to stop")
# GRAPH FUNCTION
def makeFig():
    plt.clf()
    plt.ylim(-5, 5)
    plt.title("Raspberry Pi ADS1115 Oscilloscope")
    plt.grid(True)
    plt.xlabel("Samples")
    plt.ylabel("Voltage (V)")
    plt.plot(val, 'ro-', label='Channel 0')
    plt.legend(loc='lower right')
# MAIN LOOP
while True:
    # Read voltage
    voltage = chan.voltage
    print("Channel 0: {:.4f} V".format(voltage))
    # Add value to graph
    val.append(voltage)
    # Keep last 50 samples
    if len(val) > 50:
        val.pop(0)
    # Update graph
    drawnow(makeFig)
    plt.pause(0.001)
    # Sampling delay
    time.sleep(0.05)
```

Raspberry Pi Oscilloscope in Action:

Copy the complete python code and paste in the python file we created earlier, remember we will need a monitor to view the plot so all of this should be done by either VNC or with a connected monitor or screen.

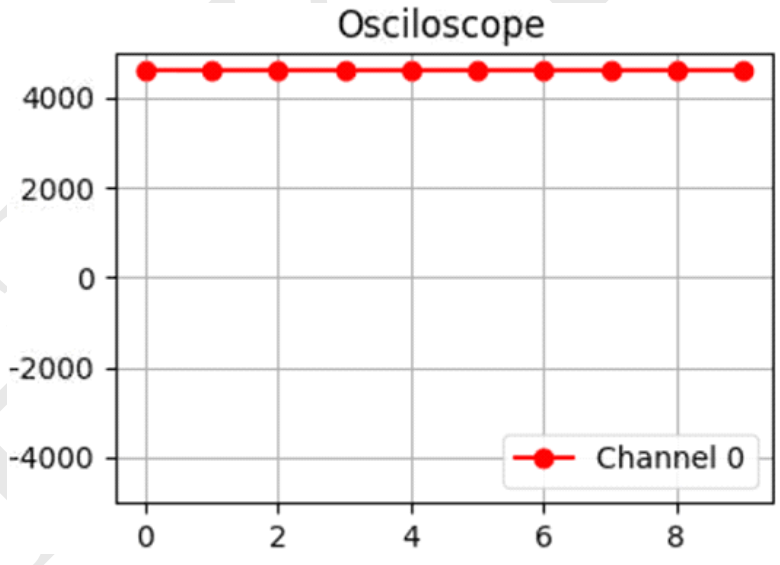
Save the code and run using;

sudo python3 scope_new.py

If you used a different name other than scope_new.py, don't forget to change this to match.

After a few seconds, you should see the ADC data being printed on the terminal.

```
pi@raspberrypi:~ $ sudo nano scope.py
pi@raspberrypi:~ $ sudo python scope.py
Reading ADS1x15 channel 0
Channel 0: 4618
/usr/lib/python2.7/dist-packages/matplotlib/backend
plotlibDeprecationWarning: Using default event loop
cific to this GUI is implemented
  warnings.warn(str, mplDeprecation)
Channel 0: 4615
Channel 0: 4616
Channel 0: 4615
Channel 0: 4614
Channel 0: 4613
Channel 0: 4614
```



You can connect an analog device like a potentiometer to a channel on the ADC and you should see the data change with each turn of the potentiometer. Or you can input Sine wave or square wave to test the output.